IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Digital Object Identifier

Two Reachability-Gated Relations in MST-wi: Temporal Mismatch, Oracle Admissibility, and Measurement Limits

QIULONG LIN[1,2] AND XIAOYU KE[1] AND NANLEI CHEN[1,2]
[1]Yango University, Fuzhou 350015, China (e-mail: ubfbzu3@163.com; 5197211@qq.com)
[2]Sogang University, Seoul 04107, Republic of Korea

Corresponding author: Nanlei Chen (e-mail: ubfbzu3@163.com).

ABSTRACT Two system-agnostic relations in MST-wi, a Web-system metamorphic testing framework, gate their verdict on a *GUI-reachability model*—a crawl-time snapshot of what the interface permits—so what they establish about object-level authorization depends on when the model was read and what the adjudicator may consult. Source inspection shows the engine records only that a relation failed: promotion to a *proven violation* is added policy, stated here explicitly, so every false-proof count is reported together with the promotion policy that produces it. On a 576-record constructed study, a legitimate access whose grant lies outside the framework's record is reported as proven in 8 of 8 alarms, and the same failure appears when the grant is made between crawl and request. Once a readable, sound and independent oracle is supplied, the relation adds no adjudication information over the direct consumer of the same oracle and outcome—an identity on the constructed grid rather than a measured gain, and conditional on an oracle that none of the systems tested here supplies, where the direct comparator in fact produces more false proofs on the two products that expose a vulnerable path and ties it at zero on the third; and the published specificity of 99.81% is an input-level figure whose denominator cannot, on its own, expose a class-uniform failure. A temporal experiment on Gitea 1.22.6 and GitLab CE 17.11.7 commits 16 authorisation transitions over 712 cross-credential cells: 29 stale false proofs appear, and a recrawl with the same credentials removes 29. A real vulnerable/fixed pair (Gitea 1.20.5 to 1.20.6) shows the independent entitlement/effect gate reporting 1 true positive on the vulnerable build, 0 false proofs, and no alarm on the fixed build—a check of the gate, not a measure of detection effectiveness; across four seeded defect classes (9 instances) the studied relation fires only on the read class, a boundary rather than a tuning gap.

INDEX TERMS BOLA/BFLA, false proof, metamorphic testing, object-level authorization, oracle problem, specificity measurement

I. INTRODUCTION

BROKEN object-level authorization (BOLA), first in the OWASP API Security Top 10 [1], is the failure to enforce, per object, that the authenticated subject is entitled to the object touched. Detecting it is an *oracle* problem: vulnerable and correct systems answer with the same success status. Metamorphic testing is an established approach to that oracle problem, and MST-wi [2] is a substantial system-agnostic instance for Web systems: 76 system-agnostic relations (MRs) over 55 Web-specific predicates, reported to detect 85% of vulnerabilities with specificity 99.81%.

This paper asks what happens to that claim when the asserted property is object-level authorization. The predicates do not express entitlement; they express what their authors call reachability—what "the functionality exposed by the user interface" permits—with the role-to-resource mapping "**not always available**" [2]. We take that sentence as premise, not discovery. Delegating adjudication to a reachability model makes the moment the model was read part of the specification, so we instantiate the predicates *as defined*—over a model recorded at crawl time—and drive the real systems through three phases: crawl, change authorization out of band, execute. The middle phase is the one the original evaluation does not exercise. The constructed study isolates the same defect without time: it seeds a share outside the framework's record and asks whether a gate that cannot see it still calls the access legal.

We ran a 576-record scenario/configuration study on a constructed system and a controlled study on Gitea 1.22.6, extended, with a methodological variation, to Gogs 0.14.3 and

GitLab 17.11.7. We propose no mechanism. What survives is three findings, one per research question (Section VI).

Research questions and findings

(1) *Temporal (RQ1).* Does the crawl-time reachability model remain sound for authorization decisions after an out-of-band authorization change? No: the adjudication as designed produces *deterministic false proofs*—a legitimate access the model cannot see is reported as a proven violation once an alarm is promoted, at rate 1.00 in the constructed study, where the share is seeded outside the framework's record, and at rate 0.40 on Gitea, where the grant is made between the crawl and the request. The predicate's two representations cannot describe either grant. The error is a property of the *reading*, not of the system. The alarm and the count of proven violations are separable, and we keep them apart: no policy changes what the relations return, while the number of contradictable claims runs from none, through 29 under promote-every-alarm, to 13 under the oracle-gated variant (Table 21). Reporting the promotion policy is therefore part of reporting the finding, not a footnote to it. (2) *Oracle (RQ2).* When a readable, sound authorization oracle fully resolves the proposition under test, does the metamorphic step add incremental adjudication information beyond the oracle and the request outcome? Not on the evaluated domain—and there the answer is a construction rather than a measurement: the direct adjudicator is built from the same entitlement and outcome quantities that define the reference label, so the two rules *cannot* part where the oracle agrees with the ledger (Section VI-C). Agreement on that domain is an identity, and we present it as one rather than as evidence that direct observation detects better. What is empirical is whether the domain is reached: the antecedent is an oracle that is readable, sound *and* admissible, and no system studied here supplies one (Section VI-F). Where it fails, substituting the direct adjudicator for the relation makes matters worse rather than better—false proofs 17 against 8 on Gogs and 18 against 5 on GitLab (Table 16)—so the practical advice is conditional on an admission test, not a blanket replacement. (3) *Measurement (RQ3).* Can a pooled specificity over follow-up inputs provide stable scenario-level resolution for a failure confined to a class? No: unaffected inputs dilute it, and a class the crawl never reaches contributes no negative inputs at all, so the figure is silent about that class (Proposition 1). The published labels are manually verified; the limit is the denominator and the input set, not the labels.

The native execution, the seeded target and the second implementation are support evidence for these findings, not further headline findings (Section VII).

What is new here, and what is not

Two of the ingredients are already established, and we neither re-derive nor claim them: that a crawl-time reachability model can go stale or under-enumerate (the predecessor attributes its residual false alarms to crawler coverage [3]), and that a reachability predicate is not an entitlement predicate (its authors say the role-to-resource mapping is ''not always available'' [2]). A third—that a pooled specificity is a weak instrument for class-level failure—is familiar from the measurement literature and is not offered as a discovery either [4]. Section IX sets each claim beside the prior position it extends and names the check that would count against it, so a reader can see which sentences carry new content. What is new is the following, and only this. (i) *A representation argument, not a coverage observation.* We show that the false proof is a deterministic consequence of which of the predicate's two representations a grant fits, hence fixed in direction, and that the variable that governs it is *when* the model was read rather than what was visible—a distinction the coverage explanation does not make, and one that also tells staleness (repairable by recrawling) from channel blindness (not repairable by the same recrawl; Section V). (ii) *The alarm/policy separation, quantified.* The technique returns an alarm; treating it as a violation is a policy we state, and we report what four policies make of the same alarms (Section VI-B). (iii) *A conditional equivalence with a named antecedent.* Promotion through the relation is a subset of direct adjudication once an admissible oracle exists, which bounds the technique's added value from above without asserting that any deployed oracle satisfies the antecedent—and we apply an explicit admission test that, on the systems tested, refuses it (Section VI-F). (iv) *A denominator argument about the published metric.* A specificity pooled over the inputs a crawl could enumerate cannot register a failure confined to a class that crawl did not reach (Proposition 1). The scope is a characterization of metamorphic testing for object-level authorization in Web/API systems, not a new detector. Although the technique is general, every empirical claim here is scoped to reachability-gated authorization relations, and we state where the transfer stops. The evidence—the three-phase instantiation with internal controls (Section III), the scenario/configuration study (Section IV), the Gitea study with an endogenous control and a seeded-vulnerability positive control extended to four defect classes (Section V), and the source reading, the native execution of the library's own relations and the second implementation (Section VII)—supports these four items rather than standing as further headline claims.

II. BACKGROUND: THE PREDICATES IN QUESTION

For authorization there is no intrinsic oracle: vulnerable and correct executions both answer with success. Metamorphic testing replaces the absent oracle with a relation over multiple executions—and the relation is only as sound as the assumptions it needs, which for authorization are exactly the facts the oracle was supposed to supply. MST-wi [2], building on Mai et al. [3], crawls a per-user model of reachable URLs and contents, then checks MRs against that recorded model. In the artifact we inspected, the predicates guarding an authorization MR read either the recorded model directly (`cannotReachThroughGUI`, `userCanRetrieveContent`) or static subject properties; we call a predicate *GUI-derived* when its value is a function of the crawl.

Why the false proof is a design consequence, not a bug

Suppose a subject can reach an object at crawl time, so the record stores it as reachable. Afterwards an administrator grants the same subject access to a *further* object through a channel—an administrative endpoint or a share link—that a crawl of the subject's own interface never traverses. At execution the guard still reads the frozen record, finds the new object absent, and concludes the subject cannot reach it; the relation then treats the successful response as a violation. Every step is exactly as specified, so no guard is miswritten: the predicate's two representations—''reachable as recorded'' and ''not reachable as recorded''—cannot express ''reachable, but only after the record was taken''. Two different cures exist for two different situations, and they must not be conflated. When the grant makes the object listable through the subject's own interface, a *re-enumeration with the same credential* picks it up: in the temporal experiment such a same-credential recrawl repairs all 29 stale cells, and no elevation of credential is involved. When instead the object never enters that crawl source at all—the granting channel is invisible to the crawled interface—no amount of re-enumerating repairs the model, and that is the boundary the Gogs residual cells in Section V-B illustrate. Observing the repair therefore requires only re-listing objects, not reading the administrative event that caused the grant, and the temporal experiment's repair claim is a same-credential one. The reported specificity does not register the failure either, because it is a property of the adjudication's *reading moment* rather than of any single generated input. This is the mechanism the rest of the paper measures, and the reason we speak of a boundary of the specification rather than a defect of an implementation.

Two census figures frame what follows. Over the 76 catalog MRs, 38 (50.0%) gate on a GUI-derived or identity predicate, and only 14 (18.42%) are authorization-relevant by name or action—an upper bound, reported as a caveat; a full-text search for predicates reading the *state of an authorization artifact* (ownership, permission record, share) finds 0 occurrences. Within the examined vocabulary, then, the authorization MRs have no authorization-artifact state observable to decide on, and the reading moment enters the specification: if an entitlement changes after the crawl, the MR evaluates reachability against a world that no longer exists. This is a statement about what the relations *mean*. Our experiments separate the two ways the world can move under the model: the constructed study varies what the gate can see, and the temporal study varies when the model was read.

III. PROTOCOL: A FAITHFUL THREE-PHASE INSTANTIATION

Two instruments carry distinct claims. The three-phase instantiation below carries the quantitative results of Sections IV and V; a direct execution of the library's own relations—source unmodified, with the build configuration and a replay harness added—carries one qualitative claim, that the published code produces the false proof we predict (Section VII-C). Neither instrument is a benchmark: the standing scope and non-claims are collected in Section VIII, and neither yields a claim of detector accuracy.

Three phases

(A) *Crawl*: record, per subject and object, whether the subject can reach the object through the interface, and freeze that record—the model the predicates will consult. (B) *Change authorization out of band*: alter a (subject, object) entitlement through a channel outside the crawled interface; the frozen model is not updated, because the architecture has no mechanism to update it. (C) *Execute the MR* against the current state while the predicates consult the frozen model. Phase B is what the original evaluation does not exercise, and it is where the reachability stand-in can be wrong (Fig. 1).

The protocol is frozen, and it was amended twice

We describe the protocol as *frozen* rather than as preregistered, because the two words make different promises and only the first one is true here. The base protocol was pinned by digest (hash `d09d904707aa`) before the first cell of the first run was observed, and the runs carry that digest in their records; but the rules a run followed are not all rules that were written before any observation. Two amendments were made. Amendment A1 demoted the secondary calibration vantage from a pass/fail gate to a recorded diagnostic, on the ground that a credential which cannot read the surface cannot distinguish ''no grant'' from ''no visibility'', so gating on that column would confound a product's choice of readable endpoints with a failure of the oracle; it scopes to the second product only, and the first product's record keeps the original gate and the original base digest. Amendment A2 records a non-local side effect of an out-of-band mutation found in an archived pilot—removing a user's group membership also removes that user's direct membership in the group's projects—and it is the reason the ledger is *not* patched with a rule learned from observation: encoding it would make E depend on a behaviour we learned from Y, which is the failure this design exists to avoid. The pilot is archived, and no cell of it enters any denominator. Which rules a given run followed is recoverable from that run's own `amendment_id` and `amendment_sha256` fields rather than from a timestamp, and we make no claim that A2 predates the first transition of the reported run: the digest is bound by the run record, whereas A2's self-reported write time falls after that run's last probe. We do not correct the field in place, because doing so would change the digest the run binds, and we state the inconsistency rather than paraphrase it away. What no amendment touched is fixed as written: the ground-truth source, the abstention exits, the *unknown* policy, the denominator conventions, and the prohibition on letting Y or O reach E. The two rules the frozen text leaves genuinely open—how an *unknown* entitlement is scored, and how an alarm becomes a claim—are treated below and in Section III-0c as explicit choices, each with its sensitivity reported (Tables 1 and 21).

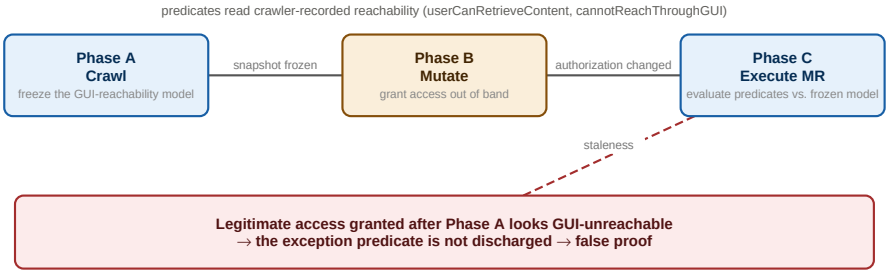


FIGURE 1. **The three-phase instantiation. The predicates consult the reachability model frozen in Phase A. A Phase B entitlement change made outside the crawled interface leaves that model stale, so in Phase C a legitimately granted access has no representation but "cannot reach"; the exception predicate is not discharged and the alarm is promoted to a proven violation—the false proof of Finding 1.**

Alarm, claim, and truth

Three objects are kept apart throughout, and no result mixes them. The *alarm* A is what the technique itself returns: for the relations studied, $A = y \wedge \neg m$, the conjunction of a successful request with the absence of the subject from the frozen reachability model. It asserts that a metamorphic relation failed; it asserts nothing about authorization, and it has no ledger, no expected decision and no verdict behind it. A *claim* H is the promotion of an alarm to "violation-proven" by a policy P that we state rather than inherit—the catalogue's purpose invites that reading, but its code does not perform it. The *truth* T is computed independently, from the ledger's entitlement E and the content-level effect Y. Promotion is therefore an adjudication policy layered on the technique's native output, not part of it, and the reported false-proof count is a property of the pair (A, P), not of A alone. Section VI-B quantifies how far the choice of P moves the published conclusion, and Section VII-C applies the same separation to the unmodified library.

Adjudicator variants and internal controls

A raw alarm becomes a verdict through an exception policy, and different policies are different methods: `raw` (the alarm itself, with no exception handling at all—this is the baseline any filter must be read against); **v1** (an *existence* filter: any sharing record on the object downgrades the alarm, regardless of whom it names or what it permits); **v2** (existence plus operation coverage: the record must cover the operation the MR triggered); **v3** (state-calibrated—consults a readable authorization introspection surface, abstains when none is readable); `degrade_all` (the floor: abstain on every alarm); `direct_only` (authorization surface only, access outcome ignored—a deliberately unfair control); and `direct_observe` (a succeeding plain request checked against the same readable surface). The last consumes the same observable quantity as the MR family and decides the negative result.

Ground truth

Every quantitative result names its ground-truth source, and it is never the detector under test. For the real-system experiments the three quantities come from different places and are kept apart: E (entitlement) is read from the runner's own ledger of authorization events, recorded at mutation time and independent of the interfaces under test; Y (effect) is the content-level outcome of a probe by the subject's own credential (for writes, owner-side re-reads and deletion re-reads rather than status codes); O (the introspection surface) is diagnostic only and never substitutes for either. That the ledger and the content probes agreed cell by cell is a measured result of the runs, not a definition, and the oracle's disagreements with the ledger are reported as findings, not absorbed. Construction state carries the constructed system and the scenario/configuration study; the pinned route tables carry the grant census; the seed list carries the seeded target.

Unknown and failure handling

Each predicate can fail to yield a bit, and the handling is fixed by the frozen protocol (Section III-0b) before any cell is observed, not negotiated per result. The failure modes are distinct and are stated as implemented. *Transport-level retries:* an HTTP request that fails to connect (status 0) is reissued up to two more times; this retry exists for the network, not for the transaction. *E undefined:* an entitlement-changing transaction that is refused or otherwise does not commit is not retried at the transaction level—the runner issues it once, and the cell is marked *failed* (reason `mutation_failed`) and excluded from every denominator, with the exclusion count reported alongside the result. *Y undefined:* an execution error or timeout leaves the cell unscored, never coerced to *allow* or *deny*—in the native engine run, 11 relations were attempted, 10 executed and 1 errored, and the errored one is reported rather than silently dropped. *O undefined:* the introspection surface may return no definite answer; the state-calibrated method abstains on those cells by construction, and the channel table reports them in an explicit *unanswered* column (56 Gitea and 72 GitLab cells for the anonymous reader, 0 on Gogs' authenticated reader) instead of absorbing them into *deny*. On those cells E and Y remain scoreable—an unanswered oracle removes the oracle-based adjudicators, not the cell. The twelve-row truth table is defined over $E \in \{\text{allow}, \text{deny}\}$; an E-undefined cell falls outside its rows and is reported as excluded, not reassigned.

The unknown *rule is a scoring choice, and both calibers are reported*

E is `unknown` when an object is absent from the ledger model. For a modelled object with no recorded granting channel the frozen text promised `unknown` (“the ledger models no channel for the triple”), while the shipped implementation returns a genuine *deny*, reserving `unknown` for the absent-object case and for the one channel the protocol singles out as unmodellable (deploy keys). We disclose this divergence rather than reconcile it silently, and we do not settle it by keeping the more favourable reading: both are reported. Closed-world default deny is what the main analysis uses; the literal reading marks those cells E-unknown and excludes them from scoring. Neither reading touches a single alarm, because every stale false proof falls on an E-allow cell: the 29 stale false proofs and the 0 live ones are identical under both. What the rule moves is the population the metrics are computed over, and it moves it far enough to change the reported rate by a factor of about four. Under default deny all 712 committed cells are scorable, the 29 false proofs are a share of 4.1% of that set, and specificity is 95.9%; under the literal rule the scorable set shrinks to the 171 cells whose entitlement the ledger fixes, the same false proofs are a share of 17.0% of it, and specificity falls to 83.0%. The literal rule is not an improvement: it raises the observed share only by discarding the 541 cells on which the adjudication happened to be right, and it leaves coverage at 24.0% of the committed cells. Precision is zero in both calibers and recall is undefined in both, because this grid contains no violation and therefore no positive; specificity and the false-proof share are reported with their denominators and are not a false discovery rate. Table 1 gives the two confusion matrices side by side.

The closed-world assumption is bounded, not verified

Default deny is sound only if the ledger enumerates every granting channel, and that assumption cannot be established from inside the run. We therefore state what it rests on and what it would cost to be wrong. It holds *by construction* for the frozen topologies used here: every grant written in the setup phase went through a modelled channel (owner, organisation owner, site administrator, public visibility, direct collaborator, team or group inheritance), and no other granting channel was exercised by the setup or mutation phase of these runs. It is also partially self-checking. An unrecorded grant that the system in fact enforces would surface as $E \neq Y$ on a probed cell, and $E = Y$ holds on all 712 of 712 cells here—a measured result rather than a definition, and not a tautology, since the oracle disagrees with E on 53 cells in the same runs. What the check cannot see is an unrecorded grant on a cell nobody probed, a channel that exists but is never exercised, or a grant the system also fails to enforce. Deployment reality adds channels this design never instantiates: directory and SSO group synchronisation, deploy keys (named by the protocol itself as unmodellable), external identity providers, and grants written straight to the database. Table 2 asks a narrower question, and answers it by intervention rather than by assertion. For each channel in turn its grant rows are deleted from every cell’s recorded basis list and the damaged entitlement is scored, with the scoring executed rather than re-derived. A cell that carries an alarm is passed to the same classifier the runs use, so its category is whatever that classifier returns. A cell that carries no alarm is counted separately as an *apparent* false negative when the damaged entitlement is deny and the observed effect is allow. The closed-form counts are asserted equal to the executed ones, so an analytic slip fails the build rather than reaching this table. Nothing a subject is permitted to do changes, because a ledger is a record and not the system: every cell that moves here remains a legitimate access, and the 29 false proofs are not repaired by its moving, only re-described. The alarm cannot move either—it is computed from the crawl snapshot, the probe response and the role, and never reads the ledger. The intervention adds one premise to that construction: deleting one channel’s rows does not cascade into another channel’s entitlement. Assertions on the 5 multi-channel cells verify that the implementation preserves an alternative recorded basis; they do not independently validate this premise. What the deletion changes is the reference label, and with it the words used for outcomes that did not change. Omitting public visibility re-labels 97 legitimate cells as deny in the damaged ledger; against that label 11 existing alarms score as *apparent* true positives and 86 legitimate accesses as *apparent* false negatives, and the reported false-proof count falls to 18. Site administration is the extreme case, where 14 of the 29 alarms become apparent true positives and no access becomes an apparent miss. Cells resting on a second channel are untouched, which is why the re-labelled counts fall short of the resting ones: 2 on public visibility, 3 on direct collaborator and 3 on team inheritance.

Read inside this analysis, the effect is one-directional and it is an undercount. With the cell sample, the alarms, the probe responses and the promotion decisions all held fixed, and only legitimate grant rows deleted under default deny, a damaged label can lower the reported false-proof count but cannot raise it. Its scope is the analysis rather than deployment: what it establishes is that the false-proof rate is conditioned on the completeness of the reference record, and that the conditioning is not benign—incompleteness hides false proofs from the measurement rather than exposing them. Under the literal rule the same cell instead leaves the denominator, the other horn of the same dilemma.

One consequence of this must be kept apart from the label argument. A promotion policy that reads the ledger instead of being fixed in advance does not merely lose its contradictions—it acts on them. The 11 cells re-labelled above are exactly the alarms a ledger-gated policy would promote once it consulted the damaged ledger, because its rule is to promote where the ledger records a definite deny. Its zero in Table 21 is a statement about a sound ledger and not a property that survives a damaged one; the two analyses touch the same cells from opposite directions and should not be merged.

Missing or modified input records fail the pipeline at entry:

TABLE 1. The 712 committed cells scored under the two *unknown* rules, with all four entries of the confusion matrix given (TP, FP, TN, FN). A positive is a violation (ledger deny, effect allow) and a negative is every other cell; in these runs $E = Y$ on every cell, so the design contains no positive and recall has no denominator (marked N/A, not 0). Specificity and precision are therefore properties of the negative set each rule admits, and that set is what changes between the two calibers: the claims do not move, the denominator does.

		cell population			confusion matrix				rates (%)	
caliber	product	scorable	negatives	E unknown	TP	FP	TN	FN	spec.	prec.
default deny	Gitea	280	280	0	0	13	267	0	95.4	0.0
default deny	GitLab	432	432	0	0	16	416	0	96.3	0.0
pooled		712	712	0	0	29	683	0	95.9	0.0
literal	Gitea	77	77	203	0	13	64	0	83.1	0.0
literal	GitLab	94	94	338	0	16	78	0	83.0	0.0
pooled		171	171	541	0	29	142	0	83.0	0.0

recall: N/A in both calibers (0 positives in the design; the temporal grid measures the false-proof face only)

TABLE 2. Scoring against a damaged reference label: what one missing granting channel does to the reported numbers. The intervention edits the ledger, not the system. The damaged reference labels are scored using the procedure described in Section III, with the cell sample, the alarms, the probe responses and the promotion decisions held fixed. Under the damaged label *re-labelled* counts the cells that now read deny; of those, *apparent TP* counts the alarmed ones (a false proof now scoring as a confirmed violation) and *apparent FN* the unalarmed ones (a legitimate access now scoring as a missed violation); *reported FP* is the false-proof count the damaged label still measures.

channel omitted	resting	re-labelled	apparent TP	apparent FN	reported FP
none (as run)	171	0	0	0	29
public visibility	99	97	11	86	18
direct collaborator	50	47	2	45	27
team / project group	11	8	2	6	27
site administrator	16	14	14	0	15

resting counts cells per channel and overlaps: 5 cells rest on two channels, so the column sums to $171 + 5$ against the 171 cells scored.

the SHA-256 manifests over inputs and outputs shipped with the artefact are checked before any generator runs, so a modified input is rejected before it can reach a table, and a missing one aborts rather than degrades; the pipeline never substitutes a default value for an absent record. The source-state facts of the native run are generated from frozen upstream inputs (commit, digests and patch manifest) rather than a live checkout, so the historical statements are recomputable from the archive alone.

Scenarios and design cells

Seven scenarios separate the reasons an MR can misfire: no shared object (S1, S2); an in-band legitimate share visible in the framework log (S3); an unshared plain violation (S4); a read-only share plus a write violation (S5); an out-of-band share invisible to the framework log (S6); an out-of-band share plus a genuine violation (S7). They are crossed with 8 configurations defined by instantiation mode, enforcement correctness, channel visibility and introspection availability. The nominal cross is 7×8 combinations, but four are inadmissible: the two no-exception configurations (`V-noE3-vuln0`, `V-noE3-vuln1`) are undefined under the two out-of-band scenarios (S6, S7), where the legitimate-sharing exception does not apply. This leaves 52 admissible design cells.

Table 3 gives the complete cross, factor by factor: the levels of each, how it is varied, the number of records at each level, and what it is crossed with. Table 26 in Section VII gives the cell-by-cell census—eight configurations by seven scenarios, with a dash wherever a combination is undefined. Two dimensions are nested rather than crossed, and we state them here instead of letting the tables imply a factorial that was not run: channel visibility exists only where the application offers a sharing facility, so the two no-exception configurations sit outside it; and the listed level was instantiated only with introspection on, so the combination *listed and unreadable* is untested rather than shown to be impossible: a product can list a share and still refuse to answer the per-object introspection query, and this enumeration would not see it.

A. WHAT THE DESIGN VARIES, AND WHAT IT CAN IDENTIFY

Because the exception policy and the reachability reading are thrown per record rather than held per configuration, every record has a partner that differs in that switch alone. Table 4 uses that: it pairs records on configuration, scenario, relation and the other switch, and reports the discordant pairs. A contrast carried by some other factor would show up as movement in the paired cells; a contrast that is not there shows up as a zero.

Three results come out of it, and they are stronger than the aggregate rates they explain. First, the reachability reading moves the outcome in one direction only: of 288 blind–aware pairs, 12 lose a false proof when the reading is aware and 0 gain one, while the same switch costs 3 detections and gains none. The aware reading is therefore a trade in a fixed direction, not noise around a rate. It is a statement

TABLE 3. The complete design. Every record is one point of the cross scenario × configuration × relation × exception policy × reachability reading; the last two are deliberately thrown switches, not replicates, so the enumeration carries no sampling error. "Records" gives the count at each level of the factor, and the last column states what the factor is crossed with. Only the listed-with-introspection cell is missing, and Section III-A says what that costs.

factor	levels	how it is varied	records	crossed with
scenario				
scenario	7	the origin of the share and the ground truth: no shared object (S1, S2); a share made through the framework's own facility (S3); an unshared violation (S4); a read-only share plus a write violation (S5); a share made out of band (S6); out of band plus a violation (S7)	48–96	all configuration levels where admissible
configuration				
instantiation	2	whether the application offers a legitimate-sharing facility at all (e3) or not (no-e3)	468 / 108	enforcement, visibility; undefined under S6–S7
enforcement	2	the application either enforces object ownership (correct) or serves the object to any authenticated subject (violating)	288 / 288	full
channel visibility	2	whether the share is written to the framework's own log (listed) or not (unlisted)	420 / 156	e3 only; no share exists to list under no-e3
introspection	2	whether the application exposes a readable per-object share list (on) or not (off)	420 / 156	*listed exists only with introspection on*
expansion				
relation	2	the metamorphic relation evaluated: MR-002 or MR-004	312 / 264	S5 carries MR-002 only
exception policy	3	how many MST-wi exception predicates are thrown before the relation fires: none (raw), one (e1), two (e1e2)	192 / 192 / 192	full
reachability reading	2	whether the gate consults a constant unreachable list (blind) or the listing current when the relation runs (aware). The two differ in what the model can see, not in when authorization changed	288 / 288	full

$44 \cdot 2 \cdot 3 \cdot 2 + 8 \cdot 3 \cdot 2 = 576$ records over 52 cells

TABLE 4. Attribution by matched pairs. Each row compares two records that differ in exactly one switch, holding configuration, scenario, relation and exception policy (or reachability reading) fixed, so the contrast cannot be carried by any other factor. "First only" and "second only" are the discordant pairs: a one-way count means the switch moves the outcome in one direction only, and a zero means it does not move it at all. The two switches that were hypothesised to matter are separated in the first two blocks; the third shows the effect is confined to violations that a readable surface can resolve.

contrast	outcome	pairs	first only	second only	both	neither	reading
blind → aware	false proof	288	12	0	24	252	the aware reading removes 12 false proofs, adds 0
blind → aware	violation detected	288	3	0	105	180	and costs 3 detections
raw → e1	false proof	192	0	0	20	172	no discordant pair: the ladder changes nothing
e1 → e1e2	false proof	192	0	0	20	172	no discordant pair: the second predicate adds nothing
unreadable → readable	verdict of violation-proven	78	0	54	0	24	all 54 changes are detections gained, none lost
unreadable → readable	verdict of violation-proven	78	0	0	0	78	no change where there is no violation to detect

about what the gate can see, not about when authorization changed: the constructed study seeds the out-of-band share before the run and never mutates entitlement inside it, so the temporal reading of this contrast belongs to Section V, not here. Second, the exception ladder has *no* discordant pair at either step: raw and e1 differ on no individual record, and neither do e1 and e1e2. That is a stronger statement than the aggregate rate not moving—20 records carry a false proof under all three policies—and it says the two exception predicates are, on this system, inert rather than merely weak. Third, oracle readability acts only where there is something to resolve: in the violating configuration 54 of 78 paired records change from `violation-proven` to `indeterminate` when the surface is switched off, none the other way; in the correct configuration 78 pairs change not at all.

What the design cannot identify follows from the two nested dimensions, and we prefer to name it. The interaction between channel visibility and readability is not estimable, because the listed-and-unreadable cell was not run; the readability contrast is identified within unlisted only, and we make no claim that readability moderates the listed channel. The legitimate-sharing mechanism is not identified under S6–S7, where it is undefined by construction, so the out-of-band estimates are for the e3 configurations alone. The two relations are not orthogonal to scenario—S5 carries MR-002 only—so we never compare relations with each other. What remains is balanced: every (exception policy, reachability reading) slice holds 52 readable and 44 unreadable records, so the readability contrast cannot be carried by either thrown switch.

Statistical treatment

The 576 records are not independent observations. Each admissible cell is expanded deterministically by the factors that are actually varied: 2 metamorphic relations (MR-002, MR-004), 3 exception-policy variants (raw, one level, two levels—the first deliberately thrown switch), and 2 reachability readings (a constant unreachable list and the listing current at execution—the second deliberately thrown switch). The second is a visibility manipulation, not a temporal one. The two relations do not both apply to every cell: only MR-002 applies under scenario S5, so 8 S5 cells carry one relation while the remaining 44 carry two. The record count is therefore

$$576 = 44 \cdot 2 \cdot 3 \cdot 2 + 8 \cdot 3 \cdot 2,$$

a deliberately unbalanced design—the per-cell multiplier differs by scenario. The records are therefore a fixed enumeration, not a sample: we report counts, rates and design-cell denominators, and we attach no inferential interval to any of them. Where a headline count has a small denominator we say so explicitly (the anchored rate, 2/2, is unstable by construction) and lead with the larger-denominator count of the same phenomenon. No conclusion rests on statistical significance. Where a comparison is paired—the ablation of Section IV-C—we report discordant-pair counts rather than an interval over a difference. The replication ceiling is explicit: one constructed system, 52 cells, 2 MRs, three real systems.

Tooling and code provenance

Development of the analysis scripts was assisted by an AI coding assistant (declared in the Declarations). Every script that assistant touched was reviewed, executed, and its output checked by the authors against the raw records; the released pipeline recomputes every reported scalar from those records, every table is generated by the pipeline, and the macro files and the two auto-generated tables share a single source of truth. No AI tool participated in the detection loop or in any adjudication.

IV. SYNTHETIC STUDY: WHERE THE ADJUDICATION GOES

This study supplies the evidence for Findings 1 and 3, and the ablation behind Finding 2; the findings themselves are stated in Section VI.

A. DEFINITIONS

Confusions are defined over the *verdict*: a *false proof* is the strict subset of false alarms promoted to `violation-proven`—an error the tool asserts as established. We also report the *detection-keep rate* (share of fired true violations still alarmed) and count only abstentions justified by an unreadable oracle.

B. WHAT THE CONSTRUCTED SYSTEM SHOWS

Attribution is clean

On the anchored configuration the false-proof rate is 2/2. With that denominator the rate is unstable by construction, so we attach to it no interval—nor a percentage, which would travel out of context. With the legitimate-sharing exception absent the rate is 0.00 (0/2), and with the channel listed it is 0.00 (0/2). The phenomenon is therefore attributable to a legitimate sharing exception the MR cannot know about, not to execution itself. A 2×2 grid over channel visibility and enforcement correctness tells the same story from the other side: on unlisted cells the false-proof rate is 44.44% with no false negatives; on listed cells it drops to 0.0% but one of the five violating records in that cell is a false negative (4 detected, 1 missed). Avoiding false proofs and keeping detection are controlled by one switch, and the switch cannot be thrown both ways at once.

Filtering exceptions does not help

The exception ladder—raw, one exception level, two—leaves the anchored rate at 1.00, 1.00, 1.00: it does not move. The exception levels we can express are themselves functions of the same reachability model, so they inherit its blind spot.

The framework cannot see the channel that matters

When a legitimate share was created through the framework's facility, its log holds 12 records and v2 produces 0 false proofs (rate 0.00); when the same share was created out of band, the log holds 0 records—no evidence the share exists—and v2's false proofs rise to 8 of 8 alarms (rate 1.00), every one promoted to `violation-proven`. The MR cannot distinguish ''this share does not exist'' from ''this share exists but was not crawled''; its only representation of the first is the second. On S7, where a genuine violation *does* sit behind the out-of-band share, the same blindness makes the alarm right for the wrong reason (6 of 6 reached)—and that coincidence of alarms with violations is what an input-level metric cannot unpack. The same tension appears within a single MR: for S3 (must not alarm) vs S5 (must alarm), no configuration in the family is correct on both—the choice of switch is a choice of *which* error to make (Fig. 2).

Stratification, not aggregation

Aggregated, v3 looks best (recovery 1.00 against `degrade_all`'s keep rate 0.00); the reading is wrong, because v3 changes regime with oracle readability. Stratified over the 36-record readable grid and its 18-record absent grid (a representative grid; Section IV-F re-runs the same comparison over all 576 records): on the readable grid it removes v2's false proofs (0 vs. 4) while still alarming on all 9 fired true violations; on the absent grid (18) it alarms on none of the 5 true violations v2 still alarms on, abstaining 13 times. Pooling converts honest abstention into apparent performance, so v3's numbers may appear only stratified.

C. FAIR ABLATION: DOES THE METAMORPHIC STEP ADD ADJUDICATION INFORMATION?

Our earlier iteration proposed v3 as a mechanism, resting on the claim that the metamorphic step was necessary. We tested that claim by removing the step and substituting the most naive consumer of the same observable. The distinc-

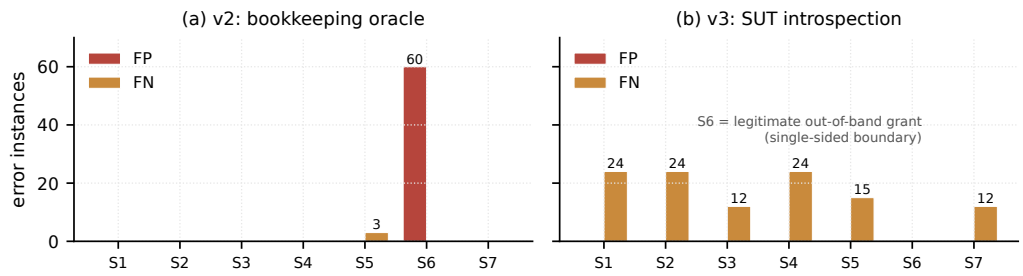


FIGURE 2. **The errors are scenario-structured. The bookkeeping oracle's false proofs concentrate on S6, the legitimate share created out of band and so invisible to the framework's own log, while the state-calibrated v3 abstains in two places: where its surface is not readable, and where a readable surface shows a covering grant it declines to override. Both panels are recomputed from the same records as Table 5.**

tion matters: `direct_only`—oracle yes, access outcome ignored—produces 108 invalid violation claims (binary false positives, none of them strict false proofs in the sense of Section III; every one falls on a request the server denied) against v3's 0 (precision 0.50), and rejecting it proves nothing because it discards half the evidence. The fair control is `direct_observe`: a succeeding ordinary request plus the same readable surface, consuming exactly what the MR-based adjudicator consumes.

Table 5 concludes the ablation. The first three rows are a ladder, and they must be read in order: with no exception handling at all, `raw` yields 120 false proofs; the existence filter v1 cuts them to 60, but pays for it by abstaining on true violations (FN rises to 18); adding operation coverage (v2) restores the true violations (FN 3) at the same false-proof count. Neither filter is therefore a free improvement, and v3's zero in the FP column is bought with the FN column. `direct_observe` has 0 false proofs against v3's 0—a tie at zero, not a win. The comparison is paired, so we report the discordance rather than an inferential interval: over the 216 true-violation records the two methods disagree on 3, and on every one of them `direct_observe` is right—it detects a violation that v3 misses or abstains on, its reachability precondition blocking a detection the plain request makes. The aggregate recalls 0.50 and 0.49 restate that asymmetry rather than measure a difference, and no margin was pre-specified, so no formal hypothesis-test claim attaches. We found no incremental adjudication advantage of the metamorphic step over direct observation in these oracle-available scenarios, and that absence provides no support for an incremental adjudication benefit in these scenarios: the step would have to be better at something the plain request cannot do. Selective presentation would have hidden the cost: v3's precision of 1.00 was bought with 111 false negatives (F1 0.65), an exchange of 60 false alarms for missed violations dressed as an improvement.

D. COVERAGE, SELECTIVE RISK, AND WHAT A POOLED PRECISION DOES NOT SAY

Table 5 scores the alarm. A method that abstains is counted as not having alarmed, so the abstention lands in TN or in FN: for the question "does the tool report a violation" that is the right scoring, but it is not the scoring for "does the method decide". Table 6 rescores the same 576 records on the decision and reports coverage, abstention rate, selective risk and request cost.

v3 decides 348 of 576 records—coverage 60.42%—and abstains on 228, 108 of them on true violations and 120 on records with none. Its precision of 1.00 and its selective risk of 0.86% are therefore properties of a 60.42% coverage: over the remaining two records in five the method claims nothing, and the pooled precision is not an error rate over all records.

The abstentions are not all of one kind (Table 6, panel b). 156 are structural—the surface returns nothing, and there is nothing for the method to consult. The other 72 fall on a *readable* surface that shows a covering grant: the access is explained, and the method still declines to assert its absence. A conservative method may prefer silence to a positive "no violation", and we do not score that as an error; but it is a non-decision, and it cannot be read as a correct rejection. The comparison with v2 makes the arithmetic visible. v2 promotes 60 false proofs; v3 declines 24 of them because the surface is unreadable and 36 because it is readable and shows the grant. Only the first 24 are the availability gate the method is named for. The rest is a false proof converted into a non-decision.

The other side of the ledger is dearer. 108 abstentions fall on true violations, and with the 3 records on which no alarm is raised at all, v3 leaves 111 of 216 violations unproven—a total that has to be assembled from two separate columns of the pooled table, the abstentions on violations and the misses, and that pooling therefore does not display as one figure.

`direct_only` is scored under its own adjudication rule, which is also the rule Table 5 uses: assert a violation whenever the authorization surface reports no granting channel, regardless of the request outcome. On the 576 records it decides 312 (the same abstention set as `direct_observe`, so the

TABLE 5. End-to-end confusion matrix over all 576 records (216 true violations, 360 correct cases). The records are a fixed enumeration, not a sample (Section III, statistical treatment), so no inferential interval is attached; where a denominator is small we say so rather than interval it. `direct_only` is a deliberately unfair control, shown to make that point.

method	TP	FP	FN	TN	precision	recall	F1
raw (no exception handling)	213	120	3	240	0.64	0.99	0.78
v1 (existence filter)	198	60	18	300	0.77	0.92	0.84
v2 (existence + coverage)	213	60	3	300	0.78	0.99	0.87
v3 (state-calibrated)	105	0	111	360	1.00	0.49	0.65
`degrade_all`	0	0	216	360	—	0.00	—
`direct_only`	108	108	108	252	0.50	0.50	0.50
`direct_observe`	108	0	108	360	1.00	0.50	0.67

two differ in rule, not in coverage) and it is right on 108 true violations. Its 108 invalid violation claims are not, however, strict false proofs in the sense defined in Section III: every one of them falls on a request the server in fact denied, so no unauthorized access resulted (108 of 108 without violation effect, 0 on allowed requests). They arise from reading the oracle's "no granting channel" as a violation claim that the request outcome contradicts, and its selective risk over the decided set is 34.62%, not zero. Keeping the rule fixed across both tables is what exposes this cost; a table that silently added the request-success condition to this row would have reported a zero risk that the method never earned. The overlap of v3's decisions with `direct_only`'s covers 240 records—the 312 records on which the direct method decides minus the 72 records on which v3 additionally declines on a readable covering surface—and the violation-detection difference on comparable denominators is stated with its own denominator, not with the 312 oracle-readable count.

`direct_observe` is scored on the same contract, and that changes what its earlier numbers meant. It abstains when the probe is unreadable, exactly as the direct adjudicator does, so it decides 312 records—coverage 54.17%—and abstains on 264, 108 of them true violations. Within the records it decides it detects 108 and raises no false proof. On the 240 records both methods decide, it detects 3 more violations than v3; the remaining difference in pooled recall is coverage, not accuracy. The last row of Table 6 is the binary version that was implicit before: it forces a negative verdict where the probe is unreadable, covers all 576 records, detects the same 108, and misses 108. Its selective risk of 18.75% is entirely missed violations, produced by the forcing rather than by the method; we report it as a reference point, not as a comparator.

Selective risk is not a ranking either, and we do not use it as one. `degrade_all`, which alarms on nothing, has a selective risk of 1.23% at 42.19% coverage and detects no violation at all; an ordering by risk alone would place it above v2. We therefore report precision and selective risk with coverage attached and never compare them across methods at unequal coverage. The stratified analysis (Section IV-F) equalises the *availability* of the oracle, not the decision rate: v3 still declines 72 readable records that show a covering grant, so the two strata do not make coverage common, and we say so wherever the comparison is made. A safe abstention is not a detection and we never count it as one.

Cost

The enumeration issues 4833 requests over 576 records (median 8 per record, range 7–10). The oracle-consuming methods add 104 probe GETs over 52 cell-operations: charged per record that is one request, and 0.18 amortised as executed, because the probe is shared by the records of a cell. `raw`, v1 and v2 issue none of their own—they read the log the crawl already wrote. We do not report wall-clock time for this study: every method replays the same recorded exchanges, so a difference would measure our replay loop rather than the system under test. The study we did execute end to end is the temporal loop of Section V: a median of 21.2 s per transition on Gitea and 7.3 s on GitLab, of which the mutation is 0.10 s and 0.09 s, the recrawl 0.38 s and 1.35 s, and the cross-credential probe sweep 20.7 s and 6.0 s. Each of these is a separate median, so the parts need not sum to the total exactly. The recrawl the mechanism needs is a small part of the loop; re-reading every credential is what costs.

E. A PUBLISHED METRIC'S RESOLUTION: WHAT A POOLED SPECIFICITY CANNOT REGISTER

The published figure is a specificity, not a false-alarm share of all inputs. MST-wi defines it as $\mathrm{TN}/(\mathrm{TN}+\mathrm{FP})$ over the follow-up inputs F, and states that each reported MT failure was manually verified [2]; the labels are therefore not the crawler's own answer. With $\mathrm{FN} = 0$ throughout that evaluation, $\mathrm{TN} = F - \mathrm{TP} - \mathrm{FP}$ and the figure equals $1 - \mathrm{FP}/(F - \mathrm{TP})$: the denominator is the *negative* follow-up inputs, not every generated input. We use that form below, and we do not attribute the published number to a circular label.

What such a figure cannot do is register a failure confined to a class, and the two reasons are arithmetic and coverage rather than labelling. First, the figure is pooled: a class contributing a share w of the negative inputs can move it by at most w, so a real class-uniform failure is diluted by unaffected inputs. Second, the follow-up inputs are those the crawl could enumerate; a class of legitimate access the crawl never reaches contributes no negative inputs and therefore cannot enter the ratio at all. The second point is the measurement face of Finding 1, and it is a statement about the coverage of the input set, not about the correctness of the human labels. We cannot reproduce the engine's exact input count, so we substitute no value for the published figure; the argument below is about the form of the denominator only.

TABLE 6. The same 576 records scored on the decision rather than on the alarm. Coverage is the share of records on which a method asserts either a violation or its absence; the abstention column gives the count and, in parentheses, its rate, split by whether the record held a true violation. *FP claims* (invalid violation claims) and *misses* are the two error counts among decided records, and *risk* is their sum over that denominator—comparable across methods only at equal coverage, which these methods do not have. A passed record is one on which the method did not fire, not a certificate of safety. `direct_only` and `direct_observe` abstain when the probe is unreadable, as the direct adjudicator does, so their coverage is 54.17%. The two differ in adjudication rule, not in scoring convention: `direct_only` asserts a violation whenever the authorization surface reports no granting channel, regardless of the request outcome; `direct_observe` additionally requires the ordinary request to succeed. All 108 FP claims of `direct_only` fall on requests the server in fact denied. These are invalid violation claims without unauthorized-access effect. Strict false proofs, defined elsewhere as promoted claims on entitled access (*E*=allow), are a different subclass; both subclasses count as incorrect claims for the binary risk in this table. The last row is a clearly labelled binary reference that forces a negative verdict on those cells, and its misses are the price of that forcing, not a property of the method. Panel (b) splits v3's abstentions by reason: only the first row is the availability gate the method is named for.

method	coverage (%)	abstained	on violation	on normal	FP claims	misses	risk (%)
`raw`	100.00	0 (0.00)	0	0	120	3	21.35
v1 (existence filter)	86.98	75 (13.02)	15	60	60	3	12.57
v2 (existence + coverage)	89.58	60 (10.42)	0	60	60	3	12.21
v3 (state-calibrated)	60.42	228 (39.58)	108	120	0	3	0.86
`degrade_all`	42.19	333 (57.81)	213	120	0	3	1.23
`direct_only`	54.17	264 (45.83)	108	156	108	0	34.62
`direct_observe`	54.17	264 (45.83)	108	156	0	0	0.00
`direct_observe`, forced negative	100.00	0 (0.00)	0	0	0	108	18.75

(b) v3's abstentions by reason

reason	abstentions (on violation / on normal)	
surface unreadable	156 (108 / 48)	nothing to consult
surface readable, covering grant	72 (0 / 72)	a false proof converted into a non-decision

A bounded formal remark on the denominator

Write $S = 1 - FP/N^-$, where $N^- = \mathrm{TN} + \mathrm{FP}$ is the number of negative follow-up inputs the run generated, and define $S_{\mathrm{scenario}} = 1 - N_{\mathrm{failed}}/N_{\mathrm{scenarios}}$, where N_{failed} is the number of scenarios in which the failure is uniform. S_{scenario} is an analytical construct for characterising resolution; it is not proposed as a replacement metric. The remark concerns resolution, not estimator consistency or power.

Proposition 1. Fix $f \geq 1$ failed scenarios, each producing a fixed number $a \geq 1$ of false-alarm inputs, and $s \geq 1$ successful scenarios each contributing m additional alarm-free negative inputs. Then $S = 1 - (fa)/(fa + sm)$, which tends to 1 as $m \to \infty$, while $S_{\mathrm{scenario}} = 1 - f/(f + s)$ is independent of m and strictly less than 1.

Proof. $N^- = fa + sm$ and $FP = fa$; substitution gives the first expression, which increases monotonically in m to 1, whereas the scenario-level figure depends only on the fixed counts f and s. □

Corollary 1. No fixed input-level specificity threshold $t < 1$ can guarantee that a scenario-uniform failure will push the metric below it: choosing $m > (fa/s) \cdot t/(1 - t)$ drives S above t while the f failing scenarios remain failing.

The finding is therefore about what the published figure can be asked to show, not about its arithmetic: it is pooled over classes, and it is computed on the inputs the crawl reached. Neither point disputes the reported value or its labels; both limit what a high value certifies about a class the run did not exercise.

Implementation robustness

An independently written implementation of the same predicates was compared on a subset fixed in advance; it bounds implementation accident only, and is reported with the confirmations in Section VII-D.

F. WHAT POOLING HID: THE ABLATION SPLIT BY ORACLE READABILITY

Table 5 pools two populations in which the oracle-consuming methods behave in opposite ways, and the pool is misleading in both directions. Section IV reports the comparison on a representative grid; this subsection, summarised in Table 8, repeats it over all 576 records. The records differ in one respect that decides whether those methods can act at all: whether the introspection surface returns a reading for the record. We therefore repeat the same comparison inside each stratum, with the same methods and the same decision rule, and we add two references that need no oracle against which the metamorphic family can be scored where no oracle exists.

Reconciling the representative grid with this partition

The 36/18 grid of Section IV is not a miniature of the 312/264 partition, and the two should not be read as one number at two sizes. The grid is a single point of the switch space: it keeps the e3 configurations, fixes the exception policy at raw and the reading at aware, and covers S3–S7, which leaves 54 records—36 readable and 18 unreadable. That starting set is the original selection, and it already contains 8 v2 false proofs; a reconciliation that started elsewhere would be a different sample with the same headcount. Table 7 closes the gap additively and record by record: the two other exception policies add 108 (72 readable), the blind reading adds 162 (108 readable), S1–S2 add 144 (96 readable), and the two

TABLE 7. Reconciling the representative grid with the full partition, record by record. The grid of Section IV is the original selection—e3 configurations, S3–S7, exception policy raw, aware reading—not a smaller version of the partition. Each block below adds records that were not already counted and the subtotal is the running union; the blocks are disjoint and their union is the full 576 records. The grid itself contains 8 v2 false proofs, so the start of the ladder is not a false-proof-free sample.

subset	readable	unreadable	records
representative grid (S3–S7, raw, aware)	36	18	54
+ other exception policies	+72	+36	+108
= both switches, S3–S7	108	54	162
+ blind reading	+108	+54	+162
= all S3–S7 records	216	108	324
+ S1–S2	+96	+48	+144
= all e3 records	312	156	468
+ no-exception configurations	+0	+108	+108
= all records	312	264	576

no-exception configurations add 108, all of them unreadable. The blocks are disjoint, their union is the full 576 records, and the readable count runs 36 → 108 → 216 → 312 → 312 (unreadable 18 → 54 → 108 → 156 → 264). The arithmetic is exact at every step, and each step was checked on record identifiers rather than on totals alone.

Stratification variable

We stratify on the *effective* readability of the surface—the value the probe returns for the record—not on the configuration switch. The distinction is not cosmetic: 108 records sit in configurations flagged `introspection=on` whose surface nevertheless returns nothing (the two no-exception configurations), so the configuration flag is not a faithful proxy for whether an oracle was available to the adjudicator, and using it would misclassify those records. The two strata partition all 576 records: 312 readable, 264 unreadable. Both are fixed enumerations, so as elsewhere we report counts and rates and attach no inferential interval.

The readable stratum strengthens Finding 2

Splitting does not rescue the metamorphic step; it removes the comparison in which it looked best. On the 312 readable records `direct_observe` reaches recall 1.00 with 0 false proofs, against v2's recall 0.97 with 36 false proofs, and against v3's recall 0.97 with 0. The ordering is therefore not an artifact of the metric—the plain request is ahead on false proofs and not behind on detections—whereas the pooled table had let the family appear competitive on recall. We do not claim domination: the comparison rests on the constructed-observable argument of Proposition 2, and `direct_observe` is not an independent method here, since it is built from the same two quantities that define the reference (its second input, the access outcome, is part of how the reference labels a violation). What the split shows is the scope of the necessity claim: where an oracle resolves the proposition, there is nothing left for the relation to add, and the pooled numbers were flattering to the relation only because they mixed in records where the oracle-bearing methods could not run.

The unreadable stratum is the relation's own regime, and it is not empty

The 264 unreadable records contain 108 true violations and 156 correct cases, a violation base rate of 40.91%. Every oracle-bearing method—v3, `direct_observe`, `direct_only`—detects none of them (0), not because the relations are weak but because those methods have nothing to consult and, by the same contract, abstain. The metamorphic family does not abstain: v2 reaches all 108 true violations (recall 1.00) with 24 false proofs, precision 0.82, a lift of 2.000× over the base rate. Against the two no-oracle references: `always_alarm` scores precision 0.41, which is the base rate itself—no information at all—and `obs_success`, which alarms on the observed success of a plain request, reaches the *same* recall at precision 0.69, lift 1.692×. The net difference between the metamorphic family and that reference is therefore not in detections (0 additional true violations) but in false alarms: 24 fewer for the metamorphic family.

What the saving consists of

The attribution is one-sided and mechanical. Over the seven scenarios, every record of the 24-alarm saving falls on S3, the legitimate share created *through the framework's own facility* and hence recorded in its log: 24 false alarms for `obs_success` against 0 for v2 on the 48 records of that scenario. On S6, the same share created out of band, the two agree exactly (24 vs. 24 false alarms on 24 records), and both for the reason developed in Section VI-A—the predicate has no representation for a grant the crawl did not see. So the incremental detection information the metamorphic step carries where no oracle exists is exactly the information the crawl-derived reachability model can see: it suppresses what was logged, and it reproduces Finding 1 on what was not. This bounds Finding 2 rather than overturning it. Finding 2's antecedent—a readable, sound oracle that fully resolves the proposition—fails on this stratum by construction, and where it fails the relation is not worthless but worth precisely the reachability model it reads: above a trivial floor, above a single uninterpreted request, and below any method that can consult the entitlement.

Limits of the comparison

Two references are deliberately weak, and we do not present them as the frontier of oracle-free detection: `obs_success` uses one request and no model, and a differential prober that compares several accounts against one another—the shape taken by the black-box line reviewed in Section IX—consumes more than `obs_success` does and was not instantiated on this record set; the same rule, evaluated on a separate controlled factorial with a live probe pair, is reported in Section VI-E. The lift we measure is therefore an estimate against weak references, not a claim about metamorphic testing versus the best oracle-free detector. Both references also share an observable with the reference labelling (`obs_success` alarms on the same access outcome that partly defines a violation), so their absolute rates carry the

TABLE 8. The ablation of Table 5 re-run inside each stratum. The two panels are not rankings of the same population but two different regimes: in the readable stratum the oracle-bearing methods can act; in the unreadable stratum they cannot act and are structurally bound to zero detections. The two reference rows need no oracle and no crawl model: `always_alarm` is the trivial floor, `obs_success` alarms on the observed success of a plain request. All numbers are counts.

stratum	method	TP	FP	FN	TN	precision	recall	F1
readable (312)	`raw`	105	72	3	132	0.59	0.97	0.74
	v1 (existence filter)	96	36	12	168	0.73	0.89	0.80
	v2 (existence+coverage)	105	36	3	168	0.74	0.97	0.84
	v3 (state-calibrated)	105	0	3	204	1.00	0.97	0.99
	`direct_observe`	108	0	0	204	1.00	1.00	1.00
	`obs_success`	108	96	0	108	0.53	1.00	0.69
	`always_alarm`	108	204	0	0	0.35	1.00	0.51
unreadable (264)	`raw`	108	48	0	108	0.69	1.00	0.82
	v1 (existence filter)	102	24	6	132	0.81	0.94	0.87
	v2 (existence+coverage)	108	24	0	132	0.82	1.00	0.90
	v3 (state-calibrated)	0	0	108	156	—	0.00	—
	`direct_observe`	0	0	108	156	—	0.00	—
	`obs_success`	108	48	0	108	0.69	1.00	0.82
	`always_alarm`	108	156	0	0	0.41	1.00	0.58

same constructiveness caveat as `direct_observe`; only the differences are meant to be read. Everything here is one constructed system, one MR family, and one criterion (the alarm) rather than the promoted verdict.

V. REAL SYSTEMS: GITEA, GOGS, GITLAB, AND A SEEDED CONTROL

This section supplies the real-system evidence for Findings 1 and 2: the Gitea study with its endogenous control, the cross-vendor replication, and the seeded positive control (support evidence, Section VII).

A synthetic system cannot tell us whether the crawl-time reading is an artifact of our construction, so we repeated the essential comparison on real systems. We chose Gitea 1.22.6 for genuine object-level sharing with per-object visibility, a machine-readable sharing surface, and a documented administrative grant path. Three scope conditions, stated before the numbers. First, on the endpoints and authorization transitions exercised here, Gitea enforced the expected decisions: no instance of the studied vulnerability was found, so the experiment measures only the false-proof side. Second, the cell study uses our instantiation of the predicates, not the engine; the engine's own relations are executed separately in Section VII-C. Third, the reachability model is reported under *both* readings—crawl-time snapshot (what the definitions presuppose) and the live execution-time state—so the time at which the model was read can be isolated as the explanatory variable. The experiment produces 96 cells, 48 per reading.

Grant surface

Table 9 counts an endpoint as grant-capable if, without modifying the system, a request can move a (subject, object) pair from not-entitled to entitled. The 8 endpoints drivable by a non-administrator are the out-of-band channel in its real form: a subject who is not an administrator can create an entitlement that a crawl of that subject's own interface need not reveal. The cross-vendor table below reports a normalized census (semantically equivalent routes merged, non-authorization grant surface excluded), which is why its Gitea figure (13) differs from the raw count (15) here; the two answer different questions.

TABLE 9. The grant surface of Gitea 1.22.6 (instance-independent).

quantity	count
paths in the system's API description	253
grant endpoints	15
of which usable without site-admin	8
revocation endpoints	2

Is the authorization surface readable, and from where?

Readability depends entirely on the vantage, so the counts below are different populations and must not be read as one denominator. From the owner/administrator vantage the sharing surface is readable on 56 of 56 owner-vantage attempts over the 48 cells: complete. From a black-box subject's own vantage—the vantage a black-box tool occupies—it is readable on 0 of 40: never. An intermediate count of 16/56 comes only from subjects who happen to be owners or administrators and does not constrain the mechanism; it is not synonymous with "an arbitrary black-box subject can read the surface". Table 10 collects these populations. On 48 of 48 applicable cells the credential-substitution check confirmed that our model agrees with the system's behavior (Fig. 3).

Is the write path verified by state rather than status?

A status code cannot establish that a write took effect, so the write path is checked by content: the subject creates a uniquely named file, the owner re-reads it byte-for-byte as the ground truth, and the file is deleted where the API allows it. Across the 3 systems, 120 of 120 cross-credential write attempts agree with the ledger, with 0 unauthorized writes; no status code contradicted the content outcome (count: 0). The owner control—a non-admin writing its own repository—changed content in 24 of 24 attempts. On 2 of the 3 systems the site administrator could not write through the contents API although the ledger grants write; those 24 control rows are excluded from the cross-credential denominator and reported as a modelling gap rather than absorbed. Restore is un-

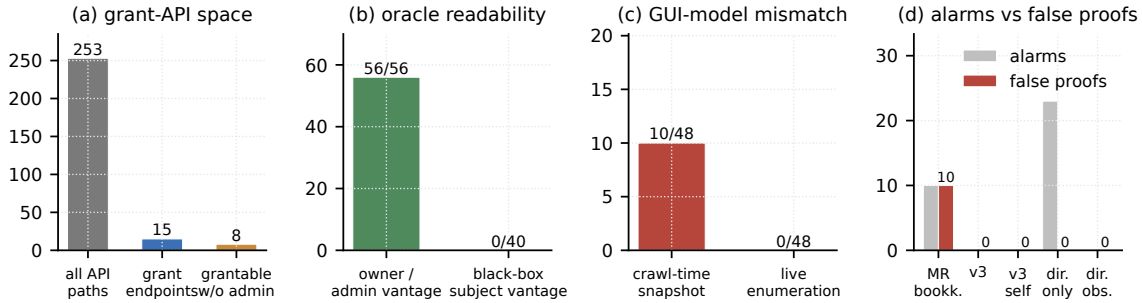


FIGURE 3. Gitea 1.22.6. (a) the grant-API space; (b) authorization-surface readability, complete from the owner/admin vantage (56/56) and never from a black-box subject's (0/40); (c) the departure from the real authorization state is present under the crawl-time reading (10/48) and absent live; (d) alarms versus false proofs per adjudicator. All counts are recomputed from the run records by the analysis pipeline.

TABLE 10. Authorization-introspection readability protocol on Gitea. Each row is a distinct population and denominator, so the rows are not comparable: both the vantage and the unit of the denominator differ.

vantage	attempts	readable
owner-vantage authorization-introspection surface	56	56
arbitrary subject (black box)	40	0
subject also owner/admin (representative grid)	56	16

TABLE 11. Departure from the real authorization state, per reading of the reachability model.

reading of the reachability model	mismatch	cells
crawl-time snapshot (as the definitions presuppose)	10	48
live execution-time state	0	48

TABLE 12. Endogenous control: two objects that are both public at the end.

object	cells	`cannotReach`	MR fires
public before the crawl	6	0	0
public only after the crawl	6	5	5

available on 1 of the 3 systems, whose contents API exposes no delete, so its probe files remain in a disposable instance.

The mismatch is in time, not in visibility

Table 11 is the load-bearing result: the adjudication departs from the real authorization state on 10 of 48 cells under the crawl-time reading and 0 of the same cells under the live reading. System state and tested relation were held fixed; only the time at which the reachability model was read changed. Under the crawl-time reading a bookkeeping adjudication produces 10 false proofs on the 25 authorized cells (rate 0.40), against 0 alarms live; v3 produces 0 false proofs, and `direct_observe` emits 0. The unfair control `direct_only` raises 23 alarms, more than any other adjudicator, simply because it ignores the request outcome; on this system none is promoted to `violation-proven` under our strict false-proof definition (0 false proofs), so alarm count and false-proof count are not the same quantity. Channel provenance of the authorized access (24 of 25 not observable by the framework log, 0.96; of these 11 are literal out-of-band grants and 13 implicit via site administration or public visibility) is a property of our topology—a design parameter, not a field prevalence. This comparison uses two snapshots and therefore cannot separate a stale model from a channel the crawl never listed; the transition experiment in Section V-A separates the two.

An endogenous control

The most serious alternative explanation is that public objects are simply harder to enumerate—visibility, not time. Two objects that end equally public but became public at different moments control for this (Table 12): the predicate reports unreachability on 5 of 6 cells only for the object made public after the crawl, and on 0 for the object public throughout, with the MR firing correspondingly (5 vs. 0). Final visibility is identical, so within this design the only manipulated explanatory variable is when the model was read.

A. EIGHT COMMITTED TRANSITIONS WITH A FROZEN MODEL, ON TWO PRODUCTS

The two-snapshot comparison above cannot separate two explanations: the model went stale, or the crawl never listed the channel in the first place. The transition experiment separates them by construction. We ran eight authorisation transitions on one fresh Gitea 1.22.6 instance and repeated the same eight, under the same frozen protocol (hash `d09d904707aa`) and the same ledger rules, on an independent GitLab CE 17.11.7 instance. The mechanisms differ per product—attaching a repository to a team becomes sharing a project with a group; removing a team member becomes removing a group member—so the two runs share a protocol, not a topology, and we report them side by side rather than pooled.

Each transition is a closed loop. We crawl every subject, serialise the per-subject reachable set, freeze it as M_0 and record its digest; commit one authorisation transaction and increment the harness version *hv* only if the transaction re-

turns 2xx; re-crawl with the *same* credentials to obtain M_1; read the oracle from the owner vantage; issue the request and verify the effect by content; and classify against the ledger at decision time. All 16 transactions committed, all 16 frozen M_0 digests were unchanged when re-checked immediately before probing, and all 16 cycles were version-consistent. Across 712 cross-credential cells (280 on Gitea plus 432 on GitLab), E and Y agree in 712 cases, so no cell is scored on a modelling discrepancy—under the implementation's closed-world default-deny reading of E; Table 15 reconciles this with the protocol's literal *unknown* rule, under which the scorable set is the 171 E-allow cells and the agreement is 171/171.

The stale model produces 29 false proofs (13 on Gitea, 16 on GitLab), and a recrawl with the same credential removes 29 of them, leaving 0 still missed and 0 live false proofs (Table 14). The composition is informative rather than uniform, and the two products agree on its shape: one cell for the direct grant, one for the inherited-channel attachment, five and six when a repository becomes public, and six and eight when a user becomes site administrator. The last two are the cases a single-object probe is least likely to anticipate, because the granting event is not about the object at all. The false-proof phenomenon is also the part of the result that the unknown-rule divergence does not touch: every stale false proof falls on an E-allow cell, so both calibers of Table 15 count the same 29, and the zero live false proofs likewise.

Revocations behave differently and must not be scored as misses. The old model still lists 17 cells whose access was revoked (7 on Gitea, 10 on GitLab). In every one of them the request is refused, so no violation occurred; treating them as false negatives would require E = deny *and* Y = allow, which never happens here.

The multi-channel transition is where the products part company, and the difference is itself a result. On Gitea, after the team member is removed, `acme/t1` remains readable through a direct collaboration while `acme/t2` becomes unreadable, and the ledger records the remaining channel. On GitLab the same test has to be moved outside the group namespace: removing a member from a group also deletes that user's project-level membership in the group's projects—the project's direct-member list went from one entry to none—so a direct grant placed inside the group would have measured that product rule instead of multi-channel access. We therefore placed the alternative channel on a personal project, and we record the cascade as a product-semantics observation rather than encoding it as a ledger rule (amendment A2 in the protocol record). This is the kind of non-local side effect that makes a hand-maintained expectation model wrong in a way no single probe can reveal.

For the role change, the read cell is unchanged while the write probe goes from HTTP 403 with unchanged content to HTTP 201 with the owner re-reading the new bytes; the owner then deletes the probe branch and observes 404, and the repository's `README` digest is unchanged after cleanup.

The oracle column separates the products more sharply than the transition counts. On Gitea the owner vantage answers 224 of 280 cells (the remainder are anonymous, for which no oracle credential exists), but the subject's own vantage answers only 7—all of them after the site-admin grant; outside it, a read collaborator receives HTTP 403 and an outsider HTTP 404. On GitLab the owner vantage answers 360 of 432, and there the subject's own credential answers 360: GitLab's member endpoint is readable by any project member. That readability is not soundness. The endpoint answers "is this principal a member", so it returns 404 for a public project a non-member may read and for a site administrator who may read everything. Measured against the ledger, the Gitea oracle disagrees on 0 cells and the GitLab oracle on 53, all of the latter in the public (44) or site-administrator (9) channel. A readable oracle can therefore be systematically wrong in exactly the channels that are cheapest to overlook, which is why we never let it set E. The two calibration regimes record this rather than hide it: 6 known cells on Gitea, and on GitLab 4 gated cells plus two recorded diagnostics for the secondary vantage (amendment A1).

A channel-by-channel view makes the mechanism explicit (Table 13), for Gitea and GitLab. Each cell is assigned to exactly one channel, and the rows sum to the cell count, so no cell is counted twice; the grouping is by the strongest channel that grants it, not a multi-label count. The same decomposition is available for Gogs (Table 17); the reason it looks unavailable there is a category error rather than a data limitation: a channel label is a property of the entitlement ledger—built from the lab topology, the documented inheritance rules and the recorded out-of-band grant events, never from a probe response—so a product whose API exposes no per-object membership route does not lose the channel axis. What such a product loses is the ability to *enumerate* those grants through that route, which is a claim about the oracle surface, not about the labelling. Both tabulated products answer every direct-collaborator and group cell correctly, and Gitea also answers its public and site-administrator cells correctly. GitLab errs only by omission, and only on the two channels that are not membership: it answers *deny* on 44 public cells that a non-member may read and on 9 site-administrator cells. Neither product returns a single false allow. A one-sided error is nevertheless sufficient to manufacture a false proof, because an alarm on such a cell is then adjudicated against a deny that the ledger does not support—which is the cross-vendor failure reported in Section V-B, now located in a named channel rather than attributed to "the oracle" as a whole.

Two properties of the run are recorded because they are what an external check can verify. The protocol was frozen before the first Gitea transition, and its digest is carried in both run records. For the amendments we do not rely on a self-reported clock: the archived pilot records only A1, and the reported GitLab run carries the digests of both A1 and A2. That content binding identifies which amendment contents the run record is associated with; it does not on its own establish when those contents were created relative to the transitions. The pilot is archived with its cells excluded from every denominator. One recorded field would, if trusted

TABLE 13. Oracle fidelity by granting channel, over the same transition cells as Table 14: a confusion matrix of the owner-vantage oracle against the ledger, per channel and per product. *allow* and *deny* count cells where the oracle agrees with the ledger; *f. allow* and *f. deny* are the two error directions; *unans.* counts cells where the surface returns no definite answer (anonymous readers). A false deny is an oracle that answers deny on a legitimately readable cell. Neither product produces a single false allow: the membership surface errs only by omission, and public and site-admin cells are exactly the channels that are not group or repository membership. The "no channel" row is the default-deny stratum: no grant of any kind is recorded for those cells, so the ledger denies them.

(a) Gitea 1.22.6

Channel	cells	allow	deny	f. allow	f. deny	unans.
site administrator	7	7	0	0	0	0
public visibility	44	35	0	0	0	9
direct collaborator	25	25	0	0	0	0
team / group	1	1	0	0	0	0
no channel	203	0	156	0	0	47
total	280	68	156	0	0	56

(b) GitLab CE 17.11.7

Channel	cells	allow	deny	f. allow	f. deny	unans.
site administrator	9	0	0	0	9	0
public visibility	53	0	0	0	44	9
direct collaborator	25	25	0	0	0	0
team / group	7	7	0	0	0	0
no channel	338	0	275	0	0	63
total	432	32	275	0	53	72

for ordering, place A2 after that run, and we do not use it for that purpose: A2's `written_at` says 02:32, later than the run's last probe at 02:26:14, so the file's own timestamp cannot establish that it was written before the first transition. We retain the field as recorded and rely on the digest carried by the run; the discrepancy is documented with the artifact rather than corrected in place. Inside a run, every cell carries its event sequence number, the harness version before and after the transition, the M_0 and M_1 digests and wall-clock timestamps, so the ordering—crawl, freeze, mutate, recrawl, oracle, probe—can be re-checked line by line rather than trusted. Within the enumeration, oracle and read probes, every call is a GET; the only mutating calls are the transitions themselves, and each is recorded with its status code. In addition to the read path, the write capable transitions carry one write probe per cycle (an HTTP POST that creates the probe artifact) followed by an owner-side DELETE that restores the state; these write calls are recorded like every other call, they change no authorization configuration, and they are the reason "all observations are GET" would be too strong a summary of the run. Failed observations are not dropped silently: a transport error after two retries is recorded as such and excluded from the scored set, a mutation outside 2xx excludes its whole transition, and an unreadable oracle is recorded as *unknown* with the cell still scored on E and Y.

Three scope limits are part of the result. First, the verdict is anchored to the request outcome, and the transitions are serialised: no mutation is issued between a cell's request and its probe, so the window between them is static by construction. A deployment in which a grant or revoke lands between the request and the adjudication is outside what these cells measure; for such a case the cell's conclusion does not apply and the observation has to be repeated, and we do not claim that the harness would detect the interleaving. Second, 0 residual is a count on these surfaces, not a completeness claim: the crawl sources themselves define what can be seen, deploy keys are outside the ledger, and none of the 16 transitions reported here commits a grant through a channel the recrawl is known to miss. That last clause is a scope statement about this sample, not about recrawling in general: the paired run below constructs such a channel deliberately, on one instance. Third, *hv* is a harness-maintained logical version, not an atomic server snapshot; the harness is single-threaded and no concurrent writer ran, which is what makes the still window meaningful. Fourth, the eight transitions are a designed set, not a sample of deployment changes, so none of the counts is a field rate.

What a recrawl repairs, and what it cannot

The 29 repairs in Table 14 are not a general remedy, and the reason is that a grant can be absent from the model for two different causes. *Staleness*: the grant is visible through the subject's own interface, was made after the snapshot was frozen, and a fresh crawl with the same credential lists it again. That is what Table 14 measures, where 29 of 29 stale false proofs disappear on recrawl and 0 resist. *Coverage*: the grant is real and enforced, but the enumeration the crawl runs does not return the object, because the view that would return it belongs to another subject or to an administrative interface—a public-visibility listing the subject's own repository list does not contain, a site-administration privilege, or a team-derived grant on a surface that does not resolve inheritance. Recrawling cannot repair a channel the crawl has no way to enumerate, and the cross-vendor run locates the residue: on Gogs 13 of 19 mismatching cells survive the live reading—5 on public visibility and 8 on the site-administrator vantage—and 5 of 11 bookkeeping false proofs survive, all 5 of them on public visibility. No team-derived or collaborator grant survives, because this product resolves team inheritance in

TABLE 14. Temporal transitions on two products under the same frozen protocol. ***Stale FP***: the frozen pre-change model does not list a resource the subject may legitimately read, so the relation would report a false proof. ***Fixed***: a recrawl with the same credential lists it again. ***Missed***: the recrawl also fails to list it. Each product has its own mechanism per row (team attach ↔ project-group share; team member removal ↔ group member removal), its own cell count (280 on Gitea, 432 on GitLab) and its own decision-time ledger; no row is pooled across products.

	Gitea 1.22.6			GitLab CE 17.11.7		
Transition	Stale FP	Fixed	Missed	Stale FP	Fixed	Missed
direct grant	1	1	0	1	1	0
direct revoke	0	0	0	0	0	0
team attach	1	1	0	1	1	0
team member remove	0	0	0	0	0	0
private→public	5	5	0	6	6	0
public→private	0	0	0	0	0	0
role read→write	0	0	0	0	0	0
site admin grant	6	6	0	8	8	0
Total	13	13	0	16	16	0

TABLE 15. The same transition cells under the two ***unknown*** rules of Section III. The implementation scores every committed cell by closed-world default deny; the protocol's literal ***unknown*** policy marks modelled-no-channel cells E-unknown and excludes them from scoring. The rule changes the scorable set, the E classification and the E/Y denominator; it does not move the stale false proofs, all of which fall on E-allow cells inside the literal-known set. Table 1 gives the rates; this table gives the counts they are computed from, product by product.

	implementation (default deny)			literal protocol (E unknown)		
Product	scored	E allow	stale FP	scorable	E/Y agree	stale FP
Gitea 1.22.6	280	77	13	77	77	13
GitLab CE 17.11.7	432	94	16	94	94	16
Total	712	—	29	171	171	29

the subject's own list. The live reading removes the staleness term and leaves the coverage term; where the same channels are represented in the subject's crawl, as on Gitea and GitLab, the live reading reaches zero.

That attribution is measured, not subtracted: in one instance, with the same subject, operation, surface and budget, and every grant made after the snapshot by another account, three channels ran in parallel—direct collaborator, team-derived, public repository (3 arms, 3 alarms). The recrawl repaired 2: the collaborator and team objects appear in the subject's list on the second pass. It left 1: the public repository, readable in the same probe as the other two (status 200 on the repository and on a file read) but not returned by the enumeration the recrawl runs—the subject's own repository list, the organisation list, and the first page of an unfiltered search. A query that names that repository does return it (1 hit), so the boundary is the enumeration's page and filter rather than the system's visibility rules: the object is reachable by a named lookup and invisible to a browsing one.

The arms are not a strictly single-variable contrast, and the departure is part of the result rather than a detail. Two of the three objects are user repositories and one is an organisation repository; more consequentially, Gogs exposes no visibility-flip route, so the public object was created public after the snapshot instead of being converted, which makes it new to the snapshot for two reasons at once—it postdates the crawl, and the crawl does not enumerate that channel. What the arm still establishes does not depend on separating those two: the object exists before the recrawl, is readable throughout it, and the recrawl still does not return it, whereas both stale arms were repaired on the same pass.

Two cautions follow. The repair count is conditioned on the transitions committed, which the three arms make concrete—2 repairs out of 3—and the temporal sample and the cross-vendor sample are not paired observations of the same cells—they differ in product, topology and mutation script—so the count surviving recrawling in one must not be subtracted from the count surviving in the other. What generalises is the mechanism rather than the ratio: a recrawl repairs exactly those grants whose granting channel the renewed crawl can enumerate, which is a property of the interface, not of the tool.

B. CROSS-VENDOR REPLICATION OF THE SNAPSHOT DESIGN: GOGS 0.14.3 AND GITLAB 17.11.7

We repeated the same three-phase protocol and 48-cell design on two further systems (Table 16): **Gogs** 0.14.3, the project Gitea forked from in 2016 (related lineage, independently evolved code; census from its pinned route table, 80 paths), and **GitLab** 17.11.7 (no lineage; census from the pinned running instance's routes, 1357 paths, since its OpenAPI document omits core grant endpoints). Gogs is a replication across implementations, GitLab across vendors within one domain.

The replication introduces a primary methodological variation: the authorization-introspection surface, which shows that *readability does not imply soundness*. Gitea's owner-vantage surface is readable and sound (agrees with effective access on all 48 cells). Gogs's collaborator-membership

TABLE 16. Cross-vendor replication under the identical three-phase protocol (crawl, out-of-band authorization change, adjudicate) on Gitea 1.22.6, Gogs 0.14.3, and GitLab 17.11.7. Counts are per reading of the same 48-cell grid; the live reading is given in parentheses where both exist. The snapshot reading is the operating point of the published relations. Gitea and Gogs share lineage (fork, 2016); GitLab shares none. The team-derived row is read out of the per-system oracle false negatives by channel (Section V-B). The Gitea grant count here is the *normalised* M1 census (semantically equivalent routes merged, non-authorization grant surface excluded), and therefore differs from the raw route count in Table 9; the two tables use different normalisations and are not in conflict.

	Gitea 1.22.6	Gogs 0.14.3	GitLab 17.11.7
API entries in pinned version (M1 denominator)	253	80	1357
Grant endpoints (M1)	13	10	18
of which usable without site admin	10	4	17
Owner-vantage oracle readable	48	48	48
oracle agrees with actual access	48	31	30
oracle wrong (false negatives)	0	17	18
oracle wrong: team-derived access missed	0	4	0
Subject-vantage oracle readable	8	35	48
Reachability mismatch, snapshot (live)	10 (0)	19 (13)	10 (0)
False proofs: bookkeeping alarm, snapshot (live)	10 (0)	11 (5)	10 (0)
False proofs: v3 (oracle-gated), snapshot (live)	0 (0)	8 (5)	5 (0)
False proofs: direct-only (oracle only)	0 (0)	17 (17)	18 (18)
False proofs: direct-observe (oracle ∧ request)	0 (0)	17 (17)	18 (18)

endpoint is equally readable (48/48) but unsound—wrong on 17 cells (35.4%), exactly the authorizations arising outside direct membership: site administration (8), public visibility (5), out-of-band grants (4). GitLab's `members/all/{u}` resolves inherited membership, but public visibility and site administration confer none: 18 false negatives (37.5%). What a surface cannot see is predictable from what it measures—why clause (2) of the admissibility condition (Section VI-F) cannot be automated from readability alone.

Table 17 locates those 17 errors instead of leaving them as a single unsoundness figure, and it does so on the ledger axis that the product's own API does not have to provide. Panel (a) groups the same 48 cells by the granting channel recorded in the ledger. The surface is correct on every cell it can represent: all 4 of the 4 direct-collaborator cells and all 27 cells with no recorded grant. It answers *deny* on all 17 cells whose entitlement arises from team membership (4), public visibility (5) or site administration (8), and it never answers allow where the ledger denies (0). The error is therefore one-sided and located: exactly the three mechanisms that the owner-vantage per-object collaboration route does not represent. Panel (b) re-groups the same cells by mechanism and shows the same concentration as a rate rather than a count, which matters because the three affected strata are small: 4 team, 5 visibility and 8 site-administrator cells, all wrong, against 4 and 27 cells, all right.

That the blind spot belongs to the chosen surface rather than to the product's whole API is recorded, not inferred: in the surface probe shipped with the run, the per-object permission suffix returns 404, the per-object team route 404 and the collaboration route the oracle consults 204, while the organisation team listing is reachable (200). A team listing is not a per-object authorization surface and a surface-gated adjudicator does not consult it; the measurement therefore reports what the gated surface licenses, not what the product could in principle expose. The decomposition is also stable across readings: the live reading of the same cells gives 31 agreements and 17 false denials, so the concentration is not an artifact of the snapshot reading.

Two labelling caveats are part of the table. First, 1 cell carries two ledger rules (site administrator and public visibility); it is counted once, under the stronger rule, so the rows sum to 48 cells rather than to the 49 recorded rules—the grouping is a partition of cells, not of mechanisms. Second, the runner's provenance bucket is a heuristic that files a collaborator grant made outside one particular repository as out-of-band; the table therefore takes its labels from the ledger, and as a cross-check the channel attribution of the 17 error cells agrees between the two sources (8/5/4). The consequence is measurable: on Gitea the surface-gated adjudicators produced no false proofs (0 v3, 0 direct-observe), while on Gogs they manufacture them (8 snapshot, 5 live, 17 direct-observe) and on GitLab again (5, 18), every v3 false proof falling on a cell whose authorization derives from site administration or public visibility (8 and 5 such cells on Gogs; 8 and 10 on GitLab). A readable membership surface with a blind spot converts that blind spot into confirmed violations; no admissible oracle was available from the tested introspection surfaces on Gogs or GitLab, so the honest adjudicator abstains on the stratum where those surfaces would license a promotion.

Two results replicate everywhere: bookkeeping alarms false-proof on legitimately granted access (Gitea 10, Gogs 11, GitLab 10 cells) with the same out-of-band-dominant grant profile; and the reachability mismatch persists (Gitea 10, Gogs 19, GitLab 10 snapshot). On Gogs the mismatch only partially resolves live (13 remain—public visibility and the site-administrator vantage never enter a subject's own list, whereas a team-derived grant does); on GitLab the live reading recovers everything (0 remain)—a second, vendor-specific failure of the snapshot premise, in the crawl model rather than the oracle.

TABLE 17. Oracle fidelity by granting channel on Gogs 0.14.3, over the same 48 snapshot cells as Table 16. Channel labels come from the entitlement ledger, which is built from the lab topology, the documented inheritance rules and the recorded out-of-band grant events, and never from a probe response; they are therefore available even though the product's own API exposes no per-object membership route that separates direct from inherited grants. *allow* and *deny* count cells where the owner-vantage introspection surface agrees with the ledger; *f. deny* is the surface answering deny on a cell the ledger allows; *unans.* counts cells with no definite answer. On this product the observed effect E coincides with the ledger entitlement on every cell, so the same table reads as $E \times O$. Panel (b) re-groups the same cells by the mechanism that produced the entitlement: the owner-vantage per-object collaboration route that the oracle consults represents direct collaborators only, so team membership, the repository visibility flag and site-administrator status are invisible to it. Whether some other endpoint in the same API could represent those grants is a separate question, and not the one a surface-gated adjudicator asks. One cell carries two ledger rules (site administrator and public visibility) and is counted once, under the stronger rule, so the rows sum to 48 cells rather than to the 49 recorded rules.

(a) by granting channel (ledger)

Channel	cells	allow	deny	f. deny	unans.
site administrator	8	0	0	8	0
public visibility	5	0	0	5	0
team / group	4	0	0	4	0
direct collaborator	4	4	0	0	0
no channel	27	0	27	0	0
total	48	4	27	17	0

(b) by granting mechanism (same cells)

Mechanism	cells	agree	f. allow	f. deny	f. deny (%)
direct collaborator (collaboration endpoint)	4	4	0	0	0.00
team membership (inherited)	4	0	0	4	100.00
repository visibility flag	5	0	0	5	100.00
site-administrator status	8	0	0	8	100.00
no grant recorded	27	27	0	0	0.00

C. A SEEDED POSITIVE CONTROL

On the tested endpoints and transitions the three real systems enforce the expected decisions, so the question remains: *when a real object-level flaw is present, does the same detector find it?* We answer with a positive control. A *controlled authorization target* faithfully emulates the Gitea API contract the engine consumes, with a correct internal authorization model and 3 seeded vulnerable instances—all instantiating one fault pattern (a none-entitlement subject served the object at `GET /api/v1/repos/{owner}/{repo}`), differing in subject and object, not in fault location; Table 18 is therefore evidence about this fault pattern, not three independent defect discoveries. Two design points. The target deliberately exposes its authorization state to the subject-side harness (mirroring the admissibility condition), which production Gitea does not—there the oracle is readable from the owner/admin vantage (56/56) but not from an arbitrary subject's (0/40)—and its introspection surface reports the *correct* `none` permission, so the oracle stays sound and the defect is visible only as a discrepancy between oracle and request. We drove it with the same topology, mutation, crawl, introspection and adjudication code as above, differing only in the system under test; ground truth comes from the seed list, not from the detector.

Table 18 is consistent with the negative result of the ablation in this controlled fault pattern: `direct_observe` and `v3` catch all 3 planted instances with zero false positives, while `direct_only`, which ignores the request outcome, raises 30 alarms on cells where the surface says `none` and no access succeeds—alarms our strict definition does not promote. The control thus narrows the claim to one fault pattern: when a surface that fully resolves the authorization question is available, the metamorphic step is not necessary for detection here. It does not generalise beyond that pattern, and it also delimits the detector: we seeded one BFLA defect (non-privileged bob succeeding at `PUT .../collaborators/{u}`, HTTP 204), which the read-oriented logic does not cover—a known boundary, reported as such rather than as a detection failure. This is a controlled research artifact, not production Gitea; a sanity check on the detector's logic, not a validation of detection effectiveness on a population, and not a statement about Gitea's CVE posture. Its source, seed list and run script are in the reproducibility package at https://github.com/ubfbuz3/JISA (tag `artifact-v1`).

TABLE 18. Positive control: detection of 3 seeded vulnerable instances (one fault pattern). TP/FN/FP/TN are counted against the known seed, not against a model.

method	TP	FN	FP	TN
`mr_bookkeeping`	3	0	2	43
`v3` (owner vantage)	3	0	0	45
`v3` (subject vantage)	3	0	0	45
`direct_only`	3	0	30	15
`direct_observe`	3	0	0	45
`direct_observe` (subject)	3	0	0	45

D. FOUR DEFECT CLASSES ON THE CONTROLLED TARGET

One fault pattern answers whether the detector finds the defect it was designed for; it does not answer whether the chain transfers across authorization defects. We therefore seed four classes on the same target—the read bypass already in use, with three instances, plus two instances each of three new classes—and drive them with the same crawl, introspection and adjudication code. C1 is the read bypass already used. C2 is a write bypass: a subject whose true permission is `none`

changes the object's metadata with `PATCH` and the change is observable afterwards. C3 is a faulty inheritance expansion: an organization member outside the team is served the team's repository, and—this is the point—the defect lives in the expansion itself, so the listing, the search results and the object read all agree that the subject may see it. C4 is an escalation: a subject that is neither owner nor administrator grants a third party access it should not have, and the grantee can then reach the object; the grant is recorded in the target's state but marked as obtained through the defect, so the correct model still answers `none`. The classes are not interchangeable: they stress different premises of the relation.

Table 19 reports per-class recall against the seeded truth and, separately, false alarms on authorized cells of the same channel. C1 is the class the gate was designed for, and it is the only class the read relation covers. C2 breaks a different channel: the relation asks whether the object is *served*, not whether it can be *changed*, so it is silent on both instances; moving the same relation to the channel the defect breaks recovers both, which says something about the relation family rather than about the relation we studied. C3 and C4 are undetected end to end, and the reason is mechanical: in both the subject already sees the object, so `cannotReachThroughGUI` is false and the antecedent of the relation does not hold. The gate is not failing to notice them; it reads its own premise and concludes there is nothing to test. They stay in the denominator—the study reports 5 of 9 for the channel-matched relation, not a rate over the classes the gate happens to cover—and the antecedent's failure is the explanation, not an exclusion.

The oracle-consuming adjudicator fires on all 9 instances once the probe is channel-matched, and on 7 of 9 when confined to the read channel. False alarms are counted per channel, because a read cell cannot stand in for a write or a grant: 23 read cells, 16 write cells and 7 grant cells, the read set shared by C1 and C3. The relation's bookkeeping variant alarms on 8 of 23, 6 of 16 and 0 of 7—the same in-band failure the paper reports elsewhere—and the oracle-based adjudicator alarms on 0 of 23, 0 of 16 and 0 of 7. Write success is judged by the object's state changing, not by the status code, so a `200` that leaves the object untouched is not counted as a detection; a grant counts only if the collaborator record actually appears. The oracle answers these recorded cases correctly; that is soundness on this target, not independence, and Section VI-F reports what the admission rule does with an oracle that is sound but not independent. Two caveats belong here. First, the target's oracle shares authorization state or decision components with the content path—the ground differs by class, and the register states it per class—so under the admission rule of Section VI-F every class is refused promotion (Section VI-F and the application register in the package): the 9 hits are recall against known truth, not independence-certified proofs. Second, these are four mechanisms on one emulator, not four discoveries in production systems. The leave-one-out is reported per method rather than as one composite number. On the channel-matched adjudicator, disabling a class's fault removes that class's alarms and leaves the other classes' alarms in place, for all 4 classes. The read relation provides the same evidence for 1 class and the channel-matched relation for 2; for the others both relations are silent whether the fault is on or off, so their leave-one-out says nothing about the relation. An alarm is thereby attributable to its own defect without implying that the relation itself covers the class.

E. A PUBLICLY FIXED DEFECT

The seeded control is a controlled artifact, so we also ran one defect that nobody here planted: Gitea 1.20.5 against 1.20.6, the release whose notes record a security fix titled "Fix comment permissions" (`#28213`, backported as `#28216`); the same defect is catalogued as CVE-2023-49946 for Forgejo, the fork that reported it. What is missing is object ownership rather than path permission: a caller entitled to the repository named in the URL can address an object that belongs to a different, private repository.

Each version runs as a fresh instance, and the collision is built into the topology: alice owns a private repository with one issue comment, bob owns a public one. Bob sends `PATCH/api/v1/repos/bob/pub/issues/comments/{alice's comment id}` with a replacement body. On 1.20.5 the write succeeds (HTTP 200) and alice's own re-read returns the replacement body, so the effect is real and not a status-code artifact. On 1.20.6 the same request is refused (HTTP 404) and alice's re-read is unchanged (Table 20). The ledger gives bob no write channel on alice's repository (E = deny), so the vulnerable cell is a true positive under the definition used throughout, and the fixed cell raises no alarm: 1 true positive and 0 false proofs across the pair. All 10 control writes behave as expected in both versions—bob's own comment is editable, the correct path is refused, a third user is refused, anonymous is refused, an absent identifier is refused—and alice's restore returns the body to the original in 2 of 2 instances. The generating script is fail-closed rather than trusting these fields: it requires the complete label set with no duplicates, checks the recovery outcome, and recomputes Y from alice's re-read of the comment body before re-running the same `classify`, refusing to emit a number if any layer disagrees.

What this establishes is narrow, and worth stating plainly. The relation is not executed on this pair: the alarm recorded here is the E/Y gate's own verdict, so the result measures the gate, not the relations' recall on a real defect. It shows that the gate can issue a correct violation claim and stops issuing it once the vendor fixes the defect, so the gate is not merely a false-alarm suppressor. It does not show that we can *find* such defects: the identifier is given, the defect is public and already fixed, and the exercise is a positive control for the gate, not a discovery. Nor is it an evaluation on a population—one defect, one fault pattern (object versus path ownership), one product line. The multi-defect coverage is not obtained from this pair but from the controlled target: Section V-D seeds read, modify, inheritance and escalation faults and reports per-class recall, with the limits of that construction stated there. The

TABLE 19. Per-class recall and per-channel false alarms of the studied chain on the controlled target. Each instance is a cell whose true (correct-model) permission is *none* and on which the defect nevertheless lets the access succeed. *read gate* is the reachability-gated relation as studied in this paper; *channel gate* moves the same relation to the channel the defect breaks; the next two columns are the oracle-consuming adjudicator on the read channel and on the channel-matched probe. False alarms are counted on authorized cells of the *same* channel—23 read, 16 write and 7 grant cells—because a read cell cannot stand in for a write or a grant: write success is judged by the object's state changing, not by the status code, and a grant only counts if the collaborator record appears. C1 and C3 share the read negatives (marked †); C2 and C4 have their own. The relation alarms on 8 of 23, 6 of 16 and 0 of 7 authorized cells, which is the in-band bookkeeping failure the paper already reports; the oracle-based adjudicator alarms on 0 of them. C1 is the class the gate was designed for. C2 breaks a different channel, so the read relation cannot see it and the channel-matched relation does. C3 and C4 falsify the antecedent—the subject can already see the object—so the gate is silent on them end to end and they remain undetected in the overall denominator; that is a mechanism explanation, not a removal from the denominator. The oracle answers these recorded cases correctly; that is soundness on this target, not independence, and no class is admitted as an independent entitlement source under the admission rule of Section VI-F. The negative blocks cost 69 read, 64 write and 42 grant requests, counted at the client rather than estimated per cell.

Class	inst.	read gate	channel gate	oracle (read)	oracle (channel)	relation FP	oracle FP
C1: read enforcement	3	3	3	3	3	8/23†	0/23†
C2: modify enforcement	2	0	2	0	2	6/16	0/16
C3: inheritance	2	0	0	2	2	8/23†	0/23†
C4: privilege escalation	2	0	0	2	2	0/7	0/7
total	9	3	5	7	9	—	—

TABLE 20. A publicly fixed defect (object-versus-path ownership). The verdict uses *E* from the ledger and *Y* from the owner's re-read, not the write status.

version	PATCH	re-read	verdict
1.20.5 (vulnerable)	200	changed	true positive
1.20.6 (fixed)	404	unchanged	no claim

claim this pair supports is about the gate's correctness on a real vulnerable/fixed pair, not about detection coverage.

VI. FINDINGS, ADMISSIBILITY, AND FALSIFICATION

Each finding is a boundary on what the published relations can establish about object-level authorization—*temporal*, *oracle* and *measurement*—carried by the studies of Sections IV–V. The confirmations they rest on (the predicates read at source, the published relations executed on a real system, an independent implementation of the same predicates) introduce no further claim and are collected as support evidence in Section VII.

A. FINDING 1 (RQ1): DETERMINISTIC FALSE PROOFS—THE TEMPORAL BOUNDARY

Taken as published—raw alarms, or alarms filtered by bookkeeping exceptions—the adjudication reports a legitimate access whose grant the crawl record does not contain as a finding of an established violation under the promotion policy P of Section III-0c, which we state explicitly for this evaluation rather than inherit, since the engine's own return value asserts nothing about authorization—in the direction forced by the two-valued predicate encoding. The two studies supply the grant in different ways: the constructed study seeds it outside the framework's record, and the temporal study makes it between the crawl and the request. On the constructed system, rate 1.00 (8 of 8 alarms) against 0 for the same share created in band; on Gitea, 10 false proofs on 25 authorized cells (0.40) under the crawl-time reading, against 0 alarms live—objects, entitlements and predicates held fixed, only the reading time changed—so the error is a property of the *reading*, not of the system; the endogenous control (Table 12) holds final visibility fixed and isolates that moment. The direction is forced: the predicate has exactly two representations, and a grant created outside the crawled interface has no representation but "cannot reach". The predecessor attributes its residual false alarms to crawler coverage [3]; we add that the failure is a function of *when* the model was read, which makes the natural oracle-based repair available while showing no incremental adjudication benefit from the metamorphic step (Finding 2).

B. HOW MUCH OF THE FALSE-PROOF COUNT IS THE PROMOTION POLICY

Finding 1 is a statement about the pair (alarm, promotion policy), and the two halves separate cleanly, because the policy is a choice while the alarm is not. Table 21 applies four policies to the same 29 alarms over the 712 scored transition cells. The alarm count is invariant: no policy changes what the relations return, and none of them recovers a violation. What a policy changes is how many alarms become claims, and therefore how many claims can be contradicted. P1 promotes every alarm and is the reading Finding 1 quantifies: 29 claims, 29 of them contradicted, none confirmed. P0 reports alarms without making claims and yields no false proofs, but it is not a better method—it establishes nothing, and its 29 withheld alarms are the same phenomenon left unadjudicated. P2, the oracle-gated policy that the state-calibrated variant implements, does not remove the phenomenon either: it makes 13 claims and all 13 of them are contradicted, on the product whose introspection surface returns false denies, while on the other product it makes no claim at all and withholds 13 alarms. P3, which promotes only where the ledger itself records a definite denial, returns zero false proofs by consulting the reference—a policy that cannot be wrong and cannot be a detector.

The reading is therefore neither "zero false proofs" nor "29 vulnerabilities". It is that the technique's native output is an alarm whose security meaning is supplied entirely by a policy the technique does not contain; that the count this paper reports is what P1 makes of those alarms; and that each zero-

false-proof variant earns its number differently—by silence (P0), by abstaining on one product while shifting the error to another source on the second (P2), or by consulting the ground truth (P3). We report the P1 count as the headline because it is the promotion the catalogue's purpose invites, not because it is the only defensible policy, and the same separation is applied to the native run in Section VII-C.

C. FINDING 2 (RQ2): NOTHING BEYOND AN ADMISSIBLE ORACLE

If the false proofs of Finding 1 are removed by consulting a readable introspection surface, it is the oracle that carries the work: for the authorization relations and scenarios evaluated here, once the oracle fully resolves the authorization proposition under test, replacing the entire metamorphic step with a single ordinary request—one that consumes the same readable oracle and access outcome—adds no adjudication information. We state the reason rather than present it as a discovered effect, and the reason is partly algebraic: on the constructed grid the agreement is an identity, because the comparator is built from the same quantities that define the reference. What does carry empirical burden is the antecedent—that such an oracle is readable, sound and independent—and the next paragraph separates the two.

We separate the two parts, because only one of them can fail.

What is construction here, and what is empirical

The comparison has one component that holds by construction and one that does not, and conflating them is the shortest path to over-reading the result. `direct_observe` claims a violation exactly when the oracle answers *deny* and the ordinary request succeeds; the reference label calls a cell a violation exactly when the ledger answers *deny* and the content-level effect answers *allow*. The two rules are built from the same two quantities, so on the domain where the oracle agrees with the ledger and the request outcome agrees with the content effect they *cannot* disagree—not because one is sharper, but because the second is the first with the reference substituted for its proxy. The agreement reported below is therefore an identity on that domain, and we report it as one: it is not independent evidence that direct observation is a superior detector, and the paired counts should be read as a characterisation of two adjudication rules rather than as a measurement of one technique against another. What is genuinely empirical is the domain: whether, on a given deployment, the oracle is readable, sound *and* admissible, and whether the request outcome tracks the content-level effect. Proposition 2 is stated conditionally for exactly that reason, and Section VI-F records the antecedent failing on every system studied here.

The algebraic part is a lemma about function composition; the empirical part is a statement about what these two relations actually read, and it has a falsifier.

The direct adjudicator. Fix, before any run, the rule D that issues `violation-proven` for (o, s, a) exactly when the oracle O answers *deny* and the ordinary request R succeeds. D consumes O and R and nothing else. It is a definition, not a result, and it is the comparator in the ablation of Section IV-C (`direct_observe`).

Lemma 1 (absorption)**.** Let a detector issue its verdict through a map $g(A, O, R)$, where A is its alarm and O, R as above. If, on the domain of cells considered, (i) A is a function $a(M, R)$ of the crawl-time model and the response only, and (ii) M is O-determined for the authorization proposition under test—that is, M carries no distinction the oracle does not already make—then there is a map g' with $g(a(M, R), O, R) = g'(O, R)$ on that domain.

Proof. By (i) the first argument of g is a function of (M, R); by (ii) M is a function of O on the domain; substituting gives $A = a'(O, R)$, so g factors through (O, R) as $g'(O, R) := g(a'(O, R), O, R)$. □

The lemma is deliberately weak: it says that *if* a detector reads nothing beyond the model and the response, its verdict cannot carry information the pair (O, R) lacks—but only when both premises (i) and (ii) hold on the domain. It does not say that any particular implementation satisfies (i) or (ii), and it does not compare two programs—it compares two functions on a domain.

Proposition 2 (what the relation can add once the oracle is admitted)**.** For the two relations examined here, premise (i) of Lemma 1 holds by source inspection: their predicates read the crawl-time reachability model and the response, and nothing else (Section VII-B). Premise (ii), by contrast, is an empirical condition on the recorded domain—M would have to be a function of O there—and we do not assume it; the conclusions below are stated so as not to need it. Write y for the response bit, m for the model bit and o for the oracle bit; the shipped alarm is $A = y \wedge \neg m$, the direct adjudicator is $D = y \wedge \neg o$, and the promoted verdict of the state-calibrated method is $H = A \wedge \neg o$, which abstains where o is undefined. On the domain where o is defined:

(a) $A = D$ if and only if $y = 0$ or $m = o$; the two agree on every record whose request fails, and part exactly on the rows with $y = 1$ and $m \neq o$;

(b) $H \Rightarrow D$, because $A \Rightarrow y$ and the two share $\neg o$. The metamorphic step therefore cannot confirm a record that the direct consumer of (y, o) does not confirm; it can only abstain or repeat.

Part (b) is algebra and needs no premise about the model. Part (a) is algebra too, but it is informative only where $m = o$, and that equality is a measured condition rather than a corollary of oracle soundness: the oracle being sound does not make the crawl-time model sound. Row 7 of Table 22 holds 14 cells with $y = 1$, $m = 0$ and $o = 1$—the oracle answers correctly, the model is stale, and the relation raises a false proof that D does not. The tempting step from "the oracle is readable and correct" to "M and O agree" is therefore invalid, and we do not take it; premise (ii) of the lemma

TABLE 21. The same 29 alarms under four promotion policies, over the 712 scored transition cells of Table 14. An alarm *A* is the relation's own output and does not depend on the policy; a *claim* is a policy's promotion of *A* to `violation-proven`; a claim is *contradicted* (a false proof) when the ledger records the access as legitimate. *withheld* counts alarms the policy declines to promote. P0 and P3 both reach zero contradicted claims and neither is evidence of a better detector: P0 makes no claim at all, and P3 decides by consulting the reference ledger. P2 halves the count and leaves it entirely on the product whose introspection surface returns false denies.

policy	alarms	claims	confirmed	contradicted	withheld
P0 report the alarm, no claim	29	0	0	0	29
P1 promote every alarm	29	29	0	29	0
P2 oracle-gated (v3)	29	13	0	13	16
of which Gitea	13	0	0	0	13
of which GitLab	16	13	0	13	3
P3 ledger-gated	29	0	0	0	29

does not hold on the recorded domain: rows 7 and 8 share $O = \text{allow}$ but carry different values of M, so O does not determine M there—which is why we do not present the relation as informationally redundant in general, only as non-additive in confirmation once a definite oracle exists. What the proposition establishes is narrower and sufficient for the argument: once a definite oracle is available, promotion through the relation is a *subset* of direct adjudication, so the relation cannot add confirmations. Whether $m = o$ holds is a measured property of each system, reported in Table 22 and in Section IV-F, not a corollary of oracle correctness.

The predicates, explicitly. The two shipped predicates read

$$A_{002} = \neg\mathrm{sup}(B, A) \ \wedge \ \neg m \\ \wedge \ \neg\mathrm{err}(o_1) \ \wedge \ q(o_1, o_2), \tag{1}$$

$$A_{004} = y \ \wedge \ \neg m, \tag{2}$$

where err is the error predicate and q normalised response identity. On the domain evaluated here $\neg\mathrm{sup} = 1$, $\mathrm{err}(o_1) = 0$, and $q = y$ in all 312 records on which the second clause was computed, so both collapse to the single map

$$A \ = \ y \ \wedge \ \neg m. \tag{3}$$

Equations (1)–(3) are not a paraphrase of the code; they reproduce the recorded alarm of all 576 constructed records and all 712 temporal cells with no disagreement (`analysis/ingest_truth.py`), and the same script checks part (a) cell by cell. Two evidence layers must be kept apart here. The temporal-cell check evaluates the domain-internal simplification of Equation (3) against the alarm each cell already recorded; it is a self-consistency check of the map, not a second, independent execution of the full MR-002, whose predicate also consults the supervisor relation and the response pair. The evidence that the full-form predicate fires as written is the constructed study's 312 predicate-level records; the native run of Section VII-C carries the unmodified library's own evaluation, and neither layer is substituted for the other. The three records on which the state-calibrated method and `direct_observe` still differ are not a counterexample to (b): all three are the same S5 write cell under the three exception settings, where the model lists the object for reading, the alarm stays silent, and the oracle reports that the write is denied. There the relation misses a violation the direct adjudicator confirms—a loss, not a gain, and one the reachability gate cannot express because it collapses read and write reachability.

Reading the table

Table 22 evaluates both verdicts on every combination of the four variables and reports how many of the 712 temporal cells fall in each row, split by the ledger's entitlement. Three things are visible in it that prose cannot state as compactly. First, the equivalence is exactly as narrow as the algebra says: the native alarm A and the direct adjudicator D—the two measured surfaces, not the promoted verdict H, which the table lists separately—agree in 530 of the 584 cells with a usable oracle and part only in the two rows where $Y = 1$ and $M \neq O$, which hold 54 cells. Second, the parting is not one phenomenon but two, and they need different remedies. In the 40 cells of row 6 the model is right and the membership surface answers *deny* on a cell that is both entitled and readable, so the direct adjudicator alone raises a false proof; a better oracle would remove those. In the 14 cells of row 7 the model is stale and the oracle right, so the relation alone raises a false proof; a recrawl would remove those. Third, row 5 shows why agreement is not a certificate of correctness: in 13 cells both surfaces answer *deny*, the ledger says the subject is entitled, the request succeeds, and both verdicts are false proofs. The proposition is about information, so it is silent on row 5—but a reader who took $A = D$ as evidence that the pair decides correctly would be wrong there, and the table says so before the reader has to notice it.

Two further readings of the same table belong here, because both bear on how far the result travels. The rows without an oracle are not idle: 128 cells return no definite answer, D is undefined on all of them, and the relation still alarms in 2—alarms that nothing in this experiment can adjudicate, which is the regime the proposition's antecedent excludes rather than a gap in it. And the model term is not free. Recomputed over the two model semantics of the constructed study, the constant variant, which alarms on every successful access, finds 108 violations at 72 false proofs; the current-listing variant finds 105 at 48—24 fewer false proofs for 3 fewer detections, over 288 records each. A relation with a reachability gate is therefore neither the alarm-free ideal of Finding 2's antecedent nor a free improvement on the ungated variant; it trades one error for the other, and the trade is the reason the gate is worth

TABLE 22. The truth table of the two relations, with the temporal cells that fall in each row. *Y*: the ordinary request succeeds. *M*: the crawl-time model lists the object for the subject. *O*: the introspection surface's answer, an em dash meaning it returns no definite answer. *A*: the alarm of either relation; on this domain both reduce to $y \wedge \neg m$. *D*: the direct adjudicator, $y \wedge \neg o$, undefined in rows 9–12. H_{v3} (abbreviated *H* below) is the state-calibrated verdict $A \wedge \neg o$. It differs from the native promotion policy *P*, whose verdict is $H_P = A$. The state-calibrated verdict exists only where *O* is definite and is therefore a subset of *D* by construction ($H{=}1 \Rightarrow D{=}1$); on the definite-oracle rows it agrees with *D* except in row 6, where *D* fires and *H* does not. The *A*/*D* columns are the two measured surfaces, and the disagreement count below is an *A*/*D* count—it must not be restated as an *H*/*D* count. Rows 9–12 leave *H* undefined like *D*. The count columns split each row by the ledger's entitlement *E*, so every row states its truth conditions for both values of *E*; a dot is an unrealised combination, and in the verdict column the first term reads *E*=deny and the second *E*=allow. Where *D* is defined the two verdicts coincide whenever *Y*=deny or *M*=*O*, and part only in the two rows with *Y*=allow and $M \neq O$—54 of the 584 readable cells. Those two rows are the two failure mechanisms that Section V-B keeps apart: in row 6 the model is right and the oracle wrong, so *D* alone raises a false proof (*A*=0, *D*=1, *H*=0); in row 7 the model is stale and the oracle right, so the relation alone does (*A*=1, *D*=0, *H*=0: the promotion suppresses the alarm, which is why row 7 must not be read as a disagreement between *H* and *D*). Row 5 is the case neither explains—both surfaces answer *deny* on a cell the ledger allows, both verdicts are false proofs, and their agreement certifies nothing.

Y	*M*	*O*	*A*	*D*	H_{v3}	verdict at the ledger	*E*=deny	*E*=allow
deny	blocked	deny	0	0	0	no access	416	·
deny	reach	deny	0	0	0	no access	15	·
deny	blocked	allow	0	0	0	no access	·	·
deny	reach	allow	0	0	0	no access	·	·
allow	blocked	deny	1	1	1	true positive / false proof	·	13
allow	reach	deny	0	1	0	miss / false proof by *D*	·	40
allow	blocked	allow	1	0	0	true positive / false proof	·	14
allow	reach	allow	0	0	0	miss / correct silence	·	86
deny	blocked	—	0	—	—	no oracle; no access	108	·
deny	reach	—	0	—	—	no oracle; no access	2	·
allow	blocked	—	1	—	—	no oracle; alarm	·	2
allow	reach	—	0	—	—	no oracle; miss / silence	·	16

stating as a variable rather than folding into the oracle.

Three differences from the ideal predicate

Equations (1)–(3) describe the predicates as written, and the shipped code departs from them in three places. None changes the algebra; each either moves cells out of the domain or corrupts a variable the two verdicts share, which is why none of them makes the relation beat *D*. (i) *The model term is a store, not a query.* `cannotReachThroughGUI` and `userCanRetrieveContent` are set membership over the crawl inputs: they issue no request and read a store loaded once. So *m* is fixed at crawl time while *y* and *o* are read at decision time, and rows 6–7 are precisely the cells on which that interval carries a change. (ii) *The error predicate is a status code.* MST-wi asks whether an error page is rendered; the port asks whether the status is at least 400. A denial served with 200, or an error page served with 403 on the subject's own object, makes *y* wrong—and *y* enters both *A* and *D*, so the two stay tied while both are wrong. This is the sense in which the tie is evidence about information and not about accuracy. (iii) *Response identity is not authorization identity.* The clause *q* compares the whole normalised response, not the authorization proposition under test; a per-viewer field would make *q* false while access had in fact been granted, and the relation would stay silent on a violation. On the evaluated domain this does not bite ($q = y$ throughout), but it is the one of the three that produces a miss rather than a false proof, and it would grow if the relation were extended to responses carrying viewer-specific fields.

The proposition is falsifiable in one specific way: exhibit an observable Z, *other than the model and the response*, that a relation consumes and that is decision-relevant. Then premise (i) fails, the lemma does not apply, and the relation could in principle beat *D*. The model itself is not such a Z: a stale model also fails to be *O*-determined, but that breaks premise (ii), and the direction of the break is visible in Table 22—row 7 is a false proof, not an advantage. We know of no such observable for these two relations, and the ablation is consistent with its absence; that is the whole of the claim. We do not assert that no such observable could exist for other relations in the catalogue, and we do not assert that the relations are useless.

The proposition concerns *adjudication* information only. It does not imply that a metamorphic relation cannot improve input generation, search-space reduction, or execution efficiency: a relation may guide which requests are issued, or which gated subset is exercised, while carrying no verdict information beyond (O, R). It is a requirement on *claims*, not a verdict on metamorphic testing: it says only that a verdict computable from (O, R) cannot be shown to beat the direct consumer of (O, R).

Scope, quantified

The comparison above is pooled. Section IV-F re-runs it inside each stratum of effective oracle readability (Table 8) and thereby bounds the claim rather than widening it. Where the oracle is readable (312 records) the conclusion is stronger than "no observed benefit": the plain request carries fewer false proofs (0 vs. 36) and no fewer detections. Where it is not (264 records) the antecedent fails—there is no oracle to consult, and every oracle-bearing method detects nothing by construction—so the metamorphic family is compared against oracle-free references instead: it retains an advantage over a trivial floor and over a single uninterpreted request ($2.000\times$ vs. $1.692\times$ over the base rate), and that advantage is exactly coextensive with what the crawl-derived model can see. Finding 2 is a claim about the regime its antecedent describes; it does not say the relations are worthless outside

it.

Evidence: Table 5—0 false proofs vs. 0 (a tie at zero, not a win) over the 216 true-violation records. The comparison is paired, so we report the discordance rather than an inferential interval: the two methods disagree on 3 records, and on every one `direct_observe` is right—the aggregate recalls 0.50 and 0.49 restate that asymmetry rather than measure a difference, and no margin was fixed in advance. Gitea independently reproduces the zero-false-proof result (0 vs. 0), but not the same comparison, because the tested surface contains no seeded vulnerability; the cross-vendor systems (Section V-B) and the seeded control (Section V-C) complete the evidence. One qualification is part of the claim: the control is *constructed*, not discovered—`direct_observe` instantiates the direct-consumer construction D of Proposition 2, so the tie is the empirical face of the necessary condition, a statement about the definitions rather than a coincidental measurement.

D. FINDING 3 (RQ3): A POOLED SPECIFICITY CANNOT REGISTER A CLASS IT DID NOT EXERCISE

The reported 99.81% is a specificity over follow-up inputs, $\mathrm{TN}/(\mathrm{TN}+\mathrm{FP})$*, not a false-alarm share of all generated inputs; in that evaluation the MT failures were manually verified, so the labels are not the crawler's own. Its denominator cannot, on its own, provide scenario-level resolution for a failure confined to a class, and the inputs it counts are the ones the crawl could enumerate.* Both halves are consistent with the reported value. The denominator: with $\mathrm{FN}=0$, the figure is $1-\mathrm{FP}/(F-\mathrm{TP})$, so a failure uniform across a class contributes only as many false positives as that class emits negative inputs and is diluted by the rest of the run (Proposition 1; Corollary 1: no fixed threshold can guarantee that such a class forces the metric below it). The coverage: if a class of legitimate access is never reached by the crawl, it contributes no negative inputs, so the ratio is silent about it even though the adjudication that consumes the crawl is not. Nor does the report fix the weights: if a failing class supplies a share w of the negative inputs, the figure moves only between 1 and $1-w$, so telling a small-share failure apart from no failure requires an estimate of w that the published number does not carry.

We are not saying the figure is false, that its labels are wrong, or that the false positives were misdiagnosed: the manual verification is exactly what makes the labels credible. The claim is narrower—that a pooled figure computed on the crawled input set does not by itself certify that a class outside that set was exercised, and that a class-uniform failure can be held above any fixed threshold by unaffected inputs.

What an endogenous count certifies, conditionally

Fix a run ω and a method D; let $n_i(D,\omega)$ be the number of negative inputs D generates for scenario i in that run and $f_i(D,\omega)$ the false proofs among them. The endogenous score is $S(D,\omega)=1-\sum_i f_i(D,\omega)\big/\sum_i n_i(D,\omega)$ when the denominator is positive. Three conditional statements, and no more. (i) For each fixed ω, the identity holds by counting: it is a definition evaluated on that run's records, so it needs no distributional assumption. (ii) Correlation among records (a design cell shares its fixture) affects what confidence one may attach to S if it were re-estimated on a fresh run; it does not move the count itself, which is why we report counts and rates and no interval. (iii) For a family of runs ω_k in which the negative-input total stays positive, $S_k \to 1$ if and only if $\sum_i f_i(D,\omega_k)\big/\sum_i n_i(D,\omega_k)\to 0$; a finite-sample $S<1$ by itself says nothing about the limit. Two arithmetic examples fix the distinction. If the false-proof count is held fixed at one while the negative-input total grows with k, then $S_k = 1-1/k \to 1$: the failure never disappears, only its weight in the ratio does. If instead the false proofs grow proportionally—$f=\alpha k$ with $0<\alpha<1$ and k chosen so the counts are integral—then $S_k=1-\alpha$ for every k: input growth produces no dilution toward 1 at all. Both mechanisms coexist with an unexercised failing class, which contributes no negative inputs and leaves every S_k unchanged (the same blind spot as Proposition 1). These are statements about one run's arithmetic, conditional on its scenario coverage; they are not a theorem that any run detects every failure, and we do not use them as one.

E. COMPETING BASELINES: WHAT A SUCCEEDING REQUEST CANNOT SEPARATE

The comparison that matters is not against our own ablations but against what a conventional differential prober can do with the same two observations. We implemented three such rules and ran them on the same 42 cells (six configurations crossed with S1–S7), each consuming one request as the owner and one as the actor. *B1 status-only* alarms whenever the replayed request returns a non-error status—the coarsest rule a replay scanner can use. *B2 equality replay* alarms only when the owner's request succeeded and the actor received byte-identical output, which is the differential rule with no authorization model. *B3* adds the strongest information a tester can hold without reading the target: it suppresses every cell covered by the tester's own share log, so it knows each grant it created itself. The fourth row is MR-002 with the v3 adjudication.

Table 23 reports the outcome. The cells contain 15 true violations, 12 legitimate accesses through a grant, and 15 denials. B1 and B2 claim on all 27 allowed cells: 15 true positives and 12 false proofs, a precision of 55.56%. On these cells the equality test adds nothing to the status test, because every allowed access returns the owner's bytes. B3 removes the six in-band grants and still issues 6 false proofs, all on S6, where the grant was seeded out of band. Ours issues 10 true positives and no false proof (precision 100.00%), at the cost of 17 abstentions: the twelve legitimate grants plus five violations in the two configurations whose authorization surface is unreadable.

Two readings of this table would be wrong. It is not a detection gain: the baselines find all fifteen violations (100.00% total recall), whereas ours confirms ten—exactly those whose surface is readable—and declines the other five. The two

recall figures for ours must be read together and are not interchangeable: on the cells where ours issues a verdict at all, recall is 100.00%, because an abstention is neither a hit nor a miss there; over all 15 violation cells, including the 5 abstained ones, total recall is 66.67%. Nor is it a free improvement: coverage falls to 59.52%. What the table isolates is the only difference the claim depends on. A succeeding request is compatible with both a grant and a defect, so a rule that treats it as proof must either ignore grants (B1, B2) or see only the grants it created (B3). The out-of-band grant is the residual, and it is not a corner case in the lab: on the Gitea snapshot the equality rule fires on all 25 legitimately allowed cells, and the in-band log still leaves 24, because the framework log does not observe 0.96 of the effective access there. That figure is a property of the constructed topology, not an estimate of field prevalence. The budget is matched at the level of the probe pair and only there: 2 requests per cell for every method, one as the owner and one as the actor, 84 shared probes in all, plus one additional read of the authorization surface for ours on the 27 cells where a relation alarms and none where it does not—0.64 per cell on average, 27 reads in total, so ours issues 111 requests against the baselines' 84. Total request count therefore differs across methods in this table, which is why it is read as a comparison of adjudication trade-offs at different information costs; the matched-total-budget comparison is reported separately below, and it does not reverse that reading. The whole comparison, both probes and the additional reads included, ran in 22.5 s of wall-clock harness time covering setup, probing, scoring and cleanup; the runner binds no CPU and measures no per-method CPU time, so we make no claim about single-core cost. Because all four methods are scored from the same two recorded responses in the same process, a per-method timing would measure our replay loop rather than the rule, so we report none.

The same comparison at a matched total budget

Table 23 matches the probe pair, not the bill: our method issues one further request per cell to read the authorization surface, so one may reasonably ask whether its precision is bought with requests the baselines were not allowed to spend. Table 24 answers that with a total-request ceiling per cell instead of a probe-pair match, on the same 42 cells, with the same accounts and the same credentials. At a ceiling of two, only the owner/actor pair fits. Our method keeps its own order rather than being forced to abstain: where the relation does not trigger it passes, which needs no surface read, and those 15 cells are recorded as true negatives; on the 27 cells where the relation does trigger, no request is left to read the surface, so it abstains rather than letting the relation stand in for the surface. That is the method under the ceiling, not a separate budget policy grafted onto it. At a ceiling of three, each method may spend one further observation of its own choosing, declared in advance—ours reads the authorization surface, a strengthened baseline B3r enumerates the actor's visible objects—while B1–B3 spend nothing extra, which is reported as under-spend rather than padded. Under that ceiling our method does not dominate: B3r reaches 14 of 15 violations with 4 false proofs, ours 10 with 0 and 17 abstentions, and ours spends 111 of the 42×3 requests the ceiling allows because the 15 untriggered cells never read the surface. An abstaining cell is not free: it pays for its probe pair and, under a ceiling that reaches the surface, for the read that made it abstain. What survives a matched budget is the trade the paper describes—a higher precision on the claims actually made, paid for in coverage where the surface cannot be read—not a detection advantage over the differential baselines, and we claim none.

F. ADMISSIBILITY, AND FALSIFICATION

Three properties must be kept apart. A surface is *readable* when its answer can be obtained from the vantage in question; *sound* when its answer equals the true authorization state; and *admissible*, for promotion to `violation-proven`, when (1) it is readable *and* (2) one can give a reason that it is not of the same kind as the defect under test—not itself a reachability artifact whose blind spot correlates with the blindness tested. Readability is what makes soundness usable, but it neither implies soundness nor confers admissibility: the owner-vantage authorization-introspection surface is readable on every cell in each system (48 Gitea, 48 Gogs, 48 GitLab), yet sound on Gitea alone—it agrees with effective access on 48, 31 and 30 of those cells respectively. The cross-vendor systems are therefore not cases of ''no oracle'': a readable membership surface exists on both, but it is unsound for the authorization channels exercised here, so it fails clause (2). Clause (2) is a per-system judgement and not automatable from readability alone; the cross-vendor results make it concrete (Gogs and GitLab fail it; their adjudicators convert the blind spots into 8 and 5 false proofs). The condition, not the technique, decides the advice. Where it holds, an analyst who already has an admissible oracle should consume it directly: on the domain where oracle and ledger coincide, promotion through the relation cannot confirm anything the oracle does not (Proposition 2(b)), so the relation adds cost without confirmations. Where it fails—which is the case on every system studied here—dropping the relation for its direct counterpart is the wrong move, and the cross-vendor numbers say why: on Gogs 0.14.3 and GitLab 17.11.7 the direct adjudicator produces 17 and 18 false proofs against 8 and 5 for the oracle-gated relation (Table 16), because the oracle there errs by omission on exactly the channels the relation's alarm is silent about. Neither product passes clause (2), and neither does Gitea: on all three, the route the oracle consults and the routes that decide the request are answered by the same process under the same permission model, so the admission rule of Section VI-F returns *reject* throughout and the advice to drop the relation is unlicensed on the systems we tested. The honest method abstains—as v3 does on the unreadable stratum, where the aggregate table would have ranked it best, a ranking that reflects abstention rather than improved detection.

TABLE 23. Competing baselines on the same 42 cells and the same probe pair. The cells comprise 15 true violations, 12 legitimate accesses through a grant (in-band or out-of-band), and 15 denials. Every method consumes one owner request and one actor request (84 shared probes); ours issues one additional read of the authorization surface on the 27 cells where a relation alarms, and none where it does not (0.64 per cell on average, 27 in total, 111 against the baselines' 84), so total requests and visible information are not matched. A false proof is a cell with E = allow that the method still reports as a proven violation. Recall is reported twice because the two figures differ exactly where a method abstains: *decided* counts only the cells the method ruled on, *all* keeps every violation cell in the denominator.

Method	Decision rule	Claims	TP	False proofs	Abstain	Coverage (%)	Recall: decided / all (%)
B1 status-only	alarm iff the actor's request returns non-error	27	15	12	0	100.00	100.00 / 100.00
B2 equality replay	alarm iff the owner succeeded and the actor received byte-identical output	27	15	12	0	100.00	100.00 / 100.00
B3 B2 + in-band log	suppress cells covered by the tester's own share log	21	15	6	0	100.00	100.00 / 100.00
Ours: MR-002 + v3	consult the SUT authorization surface; abstain if unreadable	10	10	0	17	59.52	100.00 / 66.67

TABLE 24. Equal-budget comparison on the same 42 cells and the same credentials. *cap* is the total request budget per cell. At cap 2 only the owner/actor probe pair fits: our method keeps its own order—it passes on the 15 cells where the relation does not trigger, reading no surface, and abstains on the rest, where the relation does trigger and no request is left to read the surface. At cap 3 each method may spend one further observation of its own choosing—ours reads the authorization surface, B3r enumerates the actor's visible objects—while B1–B3 spend nothing extra and are reported as under budget. No method is given padding requests. Ours therefore does not dominate: at the same ceiling B3r reaches 14 of 15 with 4 false proofs, ours 10 with 0 and 17 abstentions, and where the surface is unreadable ours refuses to claim rather than guessing. Ours spends 111 of the 42×3 requests the ceiling allows because the 15 untriggered cells never read the surface—not because an abstaining cell is free: it still pays for its probe pair, and for the surface read whenever the relation triggers under a ceiling that reaches it.

cap	method	TP	FN	FP	TN	abstain	requests
2	B1 status-only	15	0	12	15	0	84
2	B2 diff-equal	15	0	12	15	0	84
2	B3 diff+in-band	15	0	6	21	0	84
2	ours (MR + v3)	0	0	0	15	27	84
3	B1 status-only	15	0	12	15	0	84
3	B2 diff-equal	15	0	12	15	0	84
3	B3 diff+in-band	15	0	6	21	0	84
3	B3r diff+in-band+reach	14	1	4	23	0	126
3	ours (MR + v3)	10	0	0	15	17	111

An oracle-independence admission rule, applied before and after

Clause (2) needs an operational procedure, and we give one here. It is fixed as a standalone protocol—defect classes, four evidence levels, an ordered *admit*/*reject*/*unknown* rule, and what each status licenses—before the multi-class and equal-budget runs of Sections V-D and VI-E, and applied both to them and, conservatively, to the archived experiments. It is newer than the frozen temporal protocol and we do not back-date it: that protocol, pinned by digest, recorded the qualitative judgement (`oracle_independence=shared`; the oracle read for diagnosis only) but did not contain the procedure below. The rule, per target defect class (here: object-level authorization enforcement of read and write requests): enumerate, at the level the product's own documentation and observable behaviour support, the path that produces the enforcement answer y and the path that produces the oracle answer o; *admit* if neither path contains the other's decision point nor shares a component whose failure could co-occur with the tested defect; *reject* if such sharing is found; *unknown* if either path cannot be enumerated at that level, in which case independence-based promotion is refused. Applied at that level of evidence: on Gitea 1.22.6 the permission route the oracle consults and the content routes are answered by the same server binary applying the same permission model; on Gogs 0.14.3 the per-object route the oracle consults enumerates direct collaborators only, and the permission suffix that would expose the remaining channels does not exist (404), so the complete oracle cannot be completed; what that route does answer is reported as a diagnostic surface, not as a complete oracle; on GitLab 17.11.7 the `members/all` route the oracle consults and the content routes are answered by the same application consulting the same membership tables. On each product the enumeration stops at the server boundary—below it we did not audit internal source, so the finer-grained verdict would be *unknown*—and the conservative reading already available at the API level, one process and one authorization model on both paths, is recorded as *reject*. On the controlled target, where we can read the source, the finer verdict is available and is also *reject*, though the ground differs by class and the register states it per class: a shared decision function (read); a guard that duplicates the rule over the same record, so the bypass clause is guard-only, the state is shared and the rule is a copy that can drift (write); a fault the oracle does not read—which is why it reports `none` there—but whose class could sit in the inheritance branch the oracle does read (inheritance); and a shared collaborator map distinguished only by the flag the grant path writes (escalation). Two revisions of the register, logged in place and without changing a verdict, corrected

five of those grounds; the protocol and the register ship with the artifact, with an errata that corrects the protocol header's commit pointer and its Gogs version label without touching the frozen bytes. Three epistemic statuses must then be kept apart: the diagnostic *O*/*H* readings in the experiments (run under the recorded `shared` status); the classification of every promoted verdict against the independent ledger *E* and the content observation *Y* (gate correctness, which needs no oracle independence); and independence-certified proofs of previously unknown defects, which the rule refuses here. No recorded number changes under the rule; what changes is the epistemic status of the promotion, which is exactly the status the paper's claims already use.

Read together, the two findings bound what the technique's adjudication can yield for the authorization proposition studied. If the reachability stand-in is the only warrant and it is wrong, a legitimate access becomes a false proof (Finding 1). If a readable, sound oracle is instead available, the relation adds no adjudication over the direct consumer of that oracle and the outcome (Finding 2). And where no readable, admissible oracle exists—the black-box case—there is nothing for a sound method to consume, so it abstains rather than proves. Across the statuses the oracle can take, the verdict the evaluated relations can yield is thus a false proof, a result no better than the plain request (and, where the oracle is readable, worse), or an abstention. None of those is a proof of illegal access, because the relation's alarm is not evidence of it: an alarm says that the relation failed, and an ordinary request that succeeds says that the object was served. A genuine violation becomes reportable only when an independent entitlement source says *deny* and a content-level effect observation says *allow*; Section V-E reports that gate producing one true positive on a vulnerable build and none on its fixed successor. The relation contributes to neither side of that conjunction.

Each finding would fall as follows: Finding 1, if the predicates were defined over live rather than recorded reachability; Finding 2, by an authorization MR needing an observable that a succeeding request plus a readable oracle cannot produce; Finding 3, by a specificity in scenario units, or with the condition explicit, that still fails to separate the configurations studied here.

Table 25 states, for each research question, what is assumed, what was measured, and what the result does not cover. We add it because the three findings rest on different kinds of evidence—source-level functional dependence, a constructed enumeration, and two real-system studies—and a reader should be able to see which kind carries which sentence.

VII. SUPPORT EVIDENCE

Nothing here is a further finding. This section holds what the three findings rest on: the semantics of the two governing predicates read at source (Section VII-B), an execution of the published relations on a real system (Section VII-C), and the robustness of our numbers to a second, independent implementation of the same predicates (Section VII-D); the seeded positive control (Section V-C) and the cross-vendor replication (Section V-B) are the remaining confirmations and stay with the study that produced them. Each item is a confirmation of a mechanism already stated, not a new claim about what the technique can detect.

A. THE ENUMERATION, CELL BY CELL

Table 26 gives the full mapping: every configuration against every scenario, with the number of records under each populated cell and a dash where the combination is undefined. The eight configurations are the full cross of instantiation mode, enforcement correctness, channel visibility and introspection availability, less two corners. The first is undefined: with no exception mechanism there is no share to list. The second—listed with the introspection surface off—was simply not instantiated, and Section III-A treats it as untested rather than impossible. Each populated cell holds twelve records—three exception policies and two reachability readings for each of the two relations—and six where only one relation applies, as under S5.

Two partitions of these records are easy to confuse, because both are counted in the same totals. Split by relation, MR-002 holds 312 records and MR-004 264, since S5 carries the first only. Split by effective readability, 312 are readable and 264 are not, since four of the eight configurations expose a surface and four do not. The two pairs of totals coincide; the partitions do not. Section IV-F uses the second, and every statement about it can be recomputed from the census column by column.

B. CONFIRMATION AT SOURCE

Definitions can be read generously, so we checked the predicates at source in the public library [2]. It ships 23 relations, 11 authorization relations, 8 gating on the two predicates analyzed. Four implementation-level properties not made explicit in the main presentation: (1) *the freeze is hard-coded*—`cannotReachThroughGUI` is set-membership over the crawl inputs, issues no request, and is cached in a field written once and never invalidated (`userCanRetrieveContent` likewise reads a crawl-produced store loaded once); (2) *the clause that could have caught it is inert*—the "absent across all users" set is built by stopping each recorded sequence at the first login action, so it is always empty; (3) *missing data defaults to an alarm*—with an empty or null crawl argument the predicate returns "cannot reach", so an empty crawl *satisfies* the precondition: the mechanism behind Finding 1's deterministic direction; (4) *the role relation is configuration, not state*—the supervisor guard reads a static configuration file and reports true for identical accounts, so it cannot reflect a later grant.

C. EXECUTING THE PUBLISHED RELATIONS

The library ships relations but no execution environment; we supplied the system configuration and crawl snapshot and ran the relations—source unmodified, build configuration (`pom.xml`) and an external harness added (1 changed

TABLE 25. Evidence to claim. Each row names the assumption the claim needs, the artefact that carries it, and the boundary that remains. The last row is not a research question: it checks the adjudication gate used throughout, and is not evidence about detection effectiveness.

RQ	Claim	Assumption it needs	Evidence	What it does not cover
1	A crawl-time reading reports an out-of-band grant of legitimate access as a proven violation.	The grant occurs after the crawl, is recorded in the ledger, and is reachable through a channel the crawl does not enumerate.	576-record constructed study; 16 committed transitions on Gitea and GitLab, 29 stale false proofs, all 29 removed by a same-credential recrawl.	Two relations of one reachability-gated family; designed transitions, not a sample of deployment changes; no residual miss is a completeness claim.
2	Once a readable, sound oracle is supplied, the relation adds no observed adjudication benefit over the direct consumer of the same oracle and outcome.	The oracle is readable and sound on the cells scored; the relation consumes no decision-relevant observable outside (O, R).	Paired ablation on 216 true-violation records, disagreeing on 3; channel-level oracle fidelity over 712 cells.	Conditional on the antecedent: the unreadable stratum has no oracle to consult, and the oracle shares its implementation with enforcement.
3	The published specificity is an input-level figure whose denominator cannot, on its own, expose a class-uniform failure.	Scenario classes emit comparable numbers of inputs, so a uniform failure is diluted by unaffected inputs; the class shares are not reported, so the dilution is bounded rather than estimated.	Proposition 1 and its corollary; sensitivity of the example to the assumed inputs per case.	The original per-case denominators are unavailable, so the published figure is diagnosed rather than re-estimated.
Gate	An independent entitlement ledger plus a content-level effect observation reports a real defect as a true positive and stays silent on its fix.	The ledger is complete for the tested object and the effect is read back by the owner, not inferred from a status code.	Gitea 1.20.5 versus 1.20.6: 1 true positive, 0 false proofs, 0 alarms on the fixed build, 10 control probes as expected.	One publicly fixed defect found and supplied by us; this is a check of the gate, not a recall estimate.

TABLE 26. The enumeration itself: records under each configuration and scenario. A dash is an inadmissible combination, not a missing run—the no-exception configurations have no legitimate-sharing facility, so S6 and S7, which are defined by a share the facility did not record, are undefined for them. Each populated cell holds six exception-policy × reachability-reading records per relation, so twelve in all except under S5, which carries MR-002 only.

configuration	S1	S2	S3	S4	S5	S6	S7	total
`V-E3-vuln0-unlisted`	12	12	12	12	6	12	12	78
`V-E3-vuln1-unlisted`	12	12	12	12	6	12	12	78
`V-E3-vuln0-listed`	12	12	12	12	6	12	12	78
`V-E3-vuln1-listed`	12	12	12	12	6	12	12	78
`V-E3-vuln0-unlisted-nointro`	12	12	12	12	6	12	12	78
`V-E3-vuln1-unlisted-nointro`	12	12	12	12	6	12	12	78
`V-noE3-vuln0`	12	12	12	12	6	—	—	54
`V-noE3-vuln1`	12	12	12	12	6	—	—	54
records	96	96	96	96	48	72	72	576
cells	8	8	8	8	8	6	6	52

upstream file, 0 changed Java sources, two added classes totalling 612 lines)—against Gitea 1.22.6 (Table 27). 10 of 11 authorization relations executed; 2 returned failures, 6 alarms in all—the engine's own `getFailures()` value (Table 29). What the run establishes is scoped to the *reachability-gated subset that fires here*, not a full replay of the catalogue: one relation aborts on an engine-side defect, most issue no request because their own guards are unsatisfied, and the content predicate is never invoked. Almost none of the 1483.7 seconds is analysis: each leaf evaluation invokes a browser-cleanup hook costing minutes, a cost of the missing environment that bounds any reproduction.

The load-bearing cell: the subject bob holds a legitimate entitlement to alice/s1, granted out of band after the crawl. There the published `cannotReachThroughGUI` answers "cannot reach"—the answer it is *defined* to give, since the subject's crawl record has no entry for the repository;

TABLE 27. Execution facts for the native run of the published relations (Section VII-C). All values are emitted by `ingest_native_engine.py`; the upstream modification counts are generated from a frozen snapshot of the upstream source state (upstream commit, per-file digests and the build-configuration patch archived with the artifact), not from a live checkout, so they are a deterministic function of archived records; upstream Java sources are unchanged apart from the build configuration.

item	value
authorization relations: catalogue, executed, fired	11, 10, 2
engine alarms (the engine's own return value)	6
HTTP requests attributed to observing relations	270
changed upstream Java sources	0
build/config changes (one `pom.xml` line)	1
added harness classes / added lines	two / 612

across the run it answered so 1149 of 7321 invocations (15.7%), 5 on this cell, and 2 relations (OTG_AUTHZ_002, OTG_AUTHZ_002c) report a failure there. The access is au-

TABLE 28. The native alarms split into the three stages: ***A*** **(the engine's own** `getFailures()`**),** ***H*** **(the promotion policy** ***P*** **applied to** ***A*****), and** ***T*** **(independent truth from the ledger** ***E*** **and the content re-read** ***Y*****). Every row is an alarm, so** ***A*** **is not repeated as a column;** ***H*** **likewise applies to every row.** † **responder bytes identical to the owner's (sha256) and a third account refused with 404. MR names abbreviate** `OTG_AUTHZ_`***x*****.**

MR	subject	object	*E*	*Y*	*T*
002	frank	acme/t1	allow	allow†	false proof
002	bob	alice/s1	allow	allow†	false proof
002	eve	carol/s1	allow	allow†	false proof
002c	frank	acme/t1	allow	allow†	false proof
002c	bob	alice/s1	allow	allow†	false proof
002c	eve	carol/s1	allow	allow†	false proof

thorized: all 3 granted cells were re-probed, and the grantee's response hashes identically to the owner's while a third account is refused. The 6 alarms fall on 3 cells, all post-crawl grants, none on a non-entitled object. Limits: the harness's HTTP observation channel is exact for the reachability predicate (set-membership, no request) but unexercised for the content predicate (0 invocations; supervisor guard 8642, 1321 true); not all gated relations fire, for reasons inside their own guards, and 1 abort on an engine-side defect—so the reproduced false proof is that of the reachability-gated subset that fires, not a full-replay count of the published relations. And Gitea, on the tested surface, enforces the expected decisions, so the run can only exhibit false proofs.

Which stage of that claim does the native run establish?

The three stages are those defined in Section III-0c, and only the third is a security finding. Here the *native alarm A* is the engine's own return: `getFailures()` is appended by `MR.fail()` (`MR.java`:909), which logs `FAILURE` with the failing input pair and asserts nothing about authorization—no ledger, no expected decision, no verdict. The promotion policy P for this run maps every fired `OTG_AUTHZ` relation to "a violation was observed", the reading the catalogue's purpose invites and its code does not perform; since P applies to every alarm, $H = 6$ holds by construction and carries no empirical content. All of it is in the independent truth T, computed from the ledger grant (E) and a content re-read (Y: the responder's bytes are identical to the owner's and a third account is refused): 0 violations are confirmed and 6 of the 6 promoted alarms are contradicted, on 3 cells (Table 28). A reader who declines P should read the run as 6 failed metamorphic relations rather than as 6 vulnerabilities; the disagreement survives that reading, because the failures fall on accesses the ledger allows.

D. IMPLEMENTATION ROBUSTNESS

An independently written implementation of the same predicates was compared with the main one on a subset fixed before the comparison, not sampled after it: the in-band scenarios on which a second reading is defined, holding the exception-policy variant at the representative level and the reachability reading at the one each relation uses (MR-002 blind, MR-004 aware). This yields 30 (configuration, scenario) points; MR-002 is defined on 30 of the resulting relation comparisons and MR-004 on the remaining 24, giving 54 in all. The two agreed on 54 of them (1.00); the point set is a fixed enumeration, so no interval is attached. Both were written from the same reading of the semantics, so this bounds implementation accident only; it does not bound a shared blind spot, and we do not present it as corroboration.

VIII. THREATS TO VALIDITY

Source versus execution

We read the predicates at source (built under JDK 8 and Maven; Section VII-B) and then executed the relations, unmodified apart from the build configuration and our harness (Section VII-C). The harness substitutes an HTTP observation channel—exact for the reachability predicate, not exercised for the content predicate, whose invocation count we report rather than claim cleanliness we did not demonstrate. The run converts one class of claim: that a false proof is produced by the *published* relations on a real system under the promotion policy P of Section VII-C; it does not convert the rest: the scenario/configuration and cell studies remain statements about our instantiation, our exception vocabulary belongs to the same predicate family, and we report no sensitivity or specificity attributable to MST-wi.

Metric and statistics

Records within a design cell are not independent (Section III, statistical treatment) and the records are a fixed enumeration, so we report counts and rates rather than inferential intervals, and no result rests on statistical significance. The RQ2 ablation is paired and is reported as a discordance count. Finding 3 is a statement about what the published denominator pools over, not about the value of any specificity figure: we do not restate the predecessor's specificity, and no conclusion in this paper depends on it.

Two MRs, one family

We instantiated 2 MRs of one family—relations gated on the absence of evidence. The native run lifts the count to 10 of 11 catalog authorization relations but not the family: only the reachability-gated ones alarm in our run. This is the paper's single most important limitation; raising it requires relations gated otherwise, a different experiment.

Real systems

Three systems (one lineage pair, one independent), chosen for accessibility, none vulnerable on the tested surface—so only the false-proof side is measured, within one domain, the git forge. Channel proportions are topology design parameters, not field prevalence. Constructed scenarios with construction-defined ground truth are why the real-system section exists; three systems tested once each are still not a population.

Stratification and the references used in it

The split of Section IV-F is on whether the introspection surface returned a reading for the record, not on the configuration switch: 108 records sit in configurations that enable introspection but whose probe returns nothing, so the switch would misclassify them. The two oracle-free references there

TABLE 29. The published authorization relations executed against Gitea 1.22.6. Each row is the applicability chain from the catalogue to the outcome: *status* is the engine's per-relation result (10 of 11 evaluated, 1 aborted on an engine-side defect), *alarms* its own `getFailures()` return (2 relations, 6 in all), and *requests* (270 total) attributed to the relation observing them first.

MR	status	alarms	requests
OTG_AUTHZ_001	executed	0	0
OTG_AUTHZ_001b	executed	0	0
OTG_AUTHZ_001b2	executed	0	0
OTG_AUTHZ_002	executed	3	270
OTG_AUTHZ_002a	executed	0	0
OTG_AUTHZ_002b	executed	0	0
OTG_AUTHZ_002c	executed	3	0
OTG_AUTHZ_002d	executed	0	0
OTG_AUTHZ_002e	error	—	—
OTG_AUTHZ_003	executed	0	0
OTG_AUTHZ_004	executed	0	0

are deliberately weak—one unconditional, one guessing from a single request—because the point is to size the metamorphic step against methods that need nothing, not to identify the best oracle-free detector; a differential prober across accounts consumes more than those references do and is instantiated instead in the separate controlled study of Section VI-E. Both references share an observable with the labelling rule, as `direct_observe` does, so only the differences against them—not their absolute rates—are meant to be read.

Our own earlier claims

An earlier iteration proposed v3 as a mechanism on the strength of its precision (1.00), which was bought with 111 false negatives, and a control (`direct_only`, precision 0.50) that discarded the access result. Replacing it with a fair control produced the negative result. We record this as the condition any similar claim must meet: a control consuming the same observable quantity as the ground truth.

IX. RELATED WORK

Metamorphic security testing

Metamorphic testing was brought to web security by Mai et al. [3], who cast security test patterns—authentication, authorization, session—as metamorphic relations so that faults can be exposed without an oracle. The line matured into MST-wi [2], whose 76 system-agnostic relations over a Web-specific domain language are the object of our study. We take the technique's own admission that the role-to-resource mapping is "not always available" at face value and follow it to its consequence for what the adjudication can decide. A relation whose guard reads a crawl-time model is sound in the false-proof direction whenever the model does not understate the subject's reachability; equality with the authorization state is a *sufficient* condition for that, not a necessary one—a model that overstates reachability turns a legitimate access into silence rather than into a false proof, trading false proofs for misses. That equality is exactly what the technique cannot check for itself.

Detecting broken access control

A large and active literature detects access-control flaws directly, and these techniques differ in the oracle they rely on. Black-box probing drives requests with and without the rights of another subject and looks for a missing refusal [5], [6], a line that runs back to AUTHSCOPE [7] and that has been extended to deployed systems under ethical constraints [8]; where an API description is available, the oracle itself can be synthesised, for instance by transforming an OpenAPI specification into a colored Petri net that predicts per-object access [9]. A complementary, white-box line recovers the *enforced* policy from the implementation—by reverse-engineering RBAC policies [10], by statically extracting authorization policies [11] or database access-control rules [12], or by locating the permission checks themselves [13]—and then flags violations against that recovered model: BOLA in database-backed applications [14], missing-owner-check and function-level flaws in Java web applications [15], [16], function- and object-level access-control flaws across Web applications [17], and privilege escalation in polyglot microservices [18]. Specification-driven API fuzzers generate the requests well but inherit the same oracle difficulty [19]–[21]; adding explicit access-policy violation checks [22] moves them toward the detection problem but does not remove the need for a ground truth, and a recent turn delegates generation or probing to language models [23]–[25]. A concurrent line reframes BOLA under a zero-trust model, in which each request must carry an explicit authorization decision rather than one inferred from endpoint reachability [26]; our question sits at that decision level. Each of these papers reports detections against an oracle it holds privately—a policy, a seeded defect, a differential response—and none of them is our comparison class, because we do not measure detection rates. What this paper adds is three claims about one published adjudication, each stated together with the check that would count against it. Two of the ingredients are already known—that a crawl-time model can go stale, and that an authorization oracle can be unsound; the claims below differ in that the first failure is deterministic and attributable to a representation choice, the second is bounded by an explicit equivalence rather than argued case by case, and the third is a property of the published metric's denominator. First, a gate that reads a crawl-time model misreports a grant the record

does not contain, deterministically rather than intermittently (Finding 1; the constructed enumeration). Re-reading the surface repairs that misreport only when the grant is one the surface would have shown on a second reading: across the two products a same-credential recrawl removes 29 of 29 stale false proofs, leaving 0 live ones. When the grant arrived through a channel the subject's own crawl does not enumerate, re-reading that crawl does not repair it: on Gogs 13 of 19 mismatching cells persist under the live reading—5 on public visibility and 8 on the site-administrator vantage—and every one of the 5 surviving surface-gated false proofs rests on public visibility. Team-derived and collaborator grants are repaired on this product, because its own repository list resolves inheritance. Staleness is repairable and channel blindness is not, and the two call for different remedies. Second, once a readable, sound oracle is supplied, the promoted verdict is a subset of direct adjudication on the evaluated domain, so the metamorphic step adds no confirmation there (Proposition 2(b)); the check that would count against it is a record the relation confirms and the direct consumer does not. Third, the published specificity's denominator is pooled over the inputs a crawl could enumerate, so a failure confined to a class outside that set does not move it (Proposition 1). We claim no new detector, no new oracle, and no recall advantage over the access-control line; what we claim is a statement about what an oracle-substituting technique can establish, and where that statement stops. The three claims, the position each extends, what each adds and the check or counterexample attached to each are collected in Table 30.

Direct authorization oracles, and what is at stake here

Comparing responses across principals or privilege states is an established way to obtain an authorization oracle when the ground-truth relation is available, and the black-box and white-box lines above are instances of it. We propose no new direct oracle, and we do not claim that direct authorization comparison is novel. Our subject is a different question: what an existing oracle-substituting technique *decides* when it substitutes reachability for entitlement. The direct oracle enters our design in exactly one role—as the fair, same-observable control that a necessity claim about the metamorphic step must face. Without such a control, a substitution can appear necessary only because the alternative it is compared against was not allowed to consume the same decision-relevant observables.

LLMs and the oracle problem

LLMs are used for vulnerability detection and specification inference [27]–[30], including lightweight models over hybrid code features [31] and federated, explainable detectors [32]; we use none in the detection loop. The transferable condition this paper exposes is that any claim of necessity must be tested against a fair substitute that consumes the same observable quantity as the ground truth.

Measurement and evaluation

Our contribution is finally a measurement, and it connects to work on the quality of the artifacts security studies are built on—for instance the observation that vulnerability datasets are frequently mislabeled, duplicated or otherwise unfit for the claims drawn from them [4]. The artifact we report is of the same family but inside a study rather than a dataset: a specificity figure whose *denominator* is pooled over the inputs a crawl could enumerate, so it cannot register a failure confined to a class that crawl never reached, while the adjudication that consumes the same crawl can. We therefore read a testing technique's own reported metric as a measurement instrument, stating explicitly what its denominator can and cannot resolve.

X. CONCLUSION

For object-level authorization, MST-wi's predicates do not know whether the acting subject is entitled to the object; they know what the interface, as crawled, permitted. Instantiating them as defined produced three findings (Section VI).

Finding 1 (temporal)

As designed, the adjudication false-proofs on legitimately granted access, deterministically (1.00 constructed; 0.40 Gitea), because an out-of-band grant fits only the wrong one of the predicate's two representations. The alarm is the technique's own output; the count of proven violations is a policy's, and the same 29 alarms yield from 0 to 29 contradicted claims across the four policies of Table 21. On the real system the mismatch appears only under the crawl-time reading (10/48, vs. 0 live), with an endogenous control holding final visibility fixed: within this controlled design the only manipulated explanatory variable is when the model was read, not what was visible.

Finding 2 (oracle)

The natural repair—a readable, sound authorization oracle that fully resolves the proposition under test—works, and in that regime the evaluated metamorphic step provides no incremental adjudication benefit over a direct consumer of the same oracle and outcome. On the constructed grid this is an identity rather than an effect: the direct adjudicator consumes the same entitlement and outcome quantities that define the reference label, so it cannot part from it where oracle and ledger coincide, and the matched counts (false proofs 0 vs. 0; disagreement on 3 of the 216 true-violation records) characterise two rules rather than measure one technique against another. The empirical content is the antecedent—an oracle that is readable, sound *and* admissible—and it fails on every system studied here, since on each the route the oracle consults and the routes that decide the request are answered by the same process under the same permission model. Where the antecedent fails, substituting the direct adjudicator is worse, not better: 17 against 8 false proofs on Gogs and 18 against 5 on GitLab (Table 16). On Gitea the comparison is limited to false-proof behavior, because no vulnerability was present on the tested surface.

TABLE 30. Novelty ledger: each claim of this paper beside the prior position it extends, the specific increment, and the check that would count against it. Rows 1 and 4 restate ingredients that are already known and are marked as such; the increment is in what is done with them, not in their discovery. For rows 1, 3 and 4 the last column names an observation that would refute the claim. The policy row is a decomposition rather than a proposition about the world, so its entry is the reproduction that would have to fail, not a counterexample.

claim	prior position	what this paper adds	check or counterexample
A legitimate access is reported as a proven violation	residual false alarms are attributed to crawler coverage [3]; the role-to-resource mapping is ''not always available'' [2]. *Known.*	The failure is deterministic and direction-fixed by which of the predicate's two representations a grant fits; the governing variable is *when* the model was read, not what was visible; and staleness (repaired by recrawling) is separated from channel blindness (not repaired)	A grant outside the recorded model that raises no alarm, or an alarm whose direction reverses
An alarm is a violation	The catalogue's purpose invites the reading; its code performs no verdict	The promotion policy is stated as a parameter, and four policies are quantified on the same alarms, separating the technique's output from the adjudication layered on it (Table 21)	Replay the declared policies on identical alarms and independent labels; the reported policy sensitivity is unsupported if the reproduced claim counts do not differ as reported
The metamorphic step adds adjudication value once an oracle is readable	Not posed; oracle-free detection is the design premise	Promotion is a subset of direct adjudication on the domain where oracle and ledger coincide, with the antecedent made operational as an admission test that refuses every system studied here	A record the relation confirms and a direct consumer of the same oracle and outcome does not
A pooled specificity supports scenario-level resolution	Specificity is reported as an aggregate over follow-up inputs [2]. *Known to be a corpus statistic.*	The denominator argument: a class the crawl never reached contributes no negative input, so the figure is silent about that class by construction (Proposition 1)	A failure confined to a class the crawl never enumerated that nevertheless moves the reported input-unit specificity

Finding 3 (measurement)

The reported specificity of 99.81% is diagnosed as a specific published figure: it is $\mathrm{TN}/(\mathrm{TN}+\mathrm{FP})$ over the follow-up inputs the crawl could enumerate, with manually verified labels, and that denominator cannot register a failure confined to a class the run did not exercise.

Support evidence: reading the implementation confirmed the semantics and added the mechanism—the crawl model is frozen by a cache never invalidated, and a missing crawl defaults to ''unreachable''—and the positive control is a sanity check that closes the loop: on a seeded vulnerable target the surface-gated adjudicator detects all planted instances of the fault pattern it was built for with zero false positives, and on four seeded defect classes it detects the read bypass and nothing else—the write bypass breaks a channel the relation does not test, and the inheritance and escalation faults falsify the antecedent because the subject can already see the object.

What the two added experiments carry

The temporal finding is not confined to one product. Under the same frozen protocol, whose two amendments are named and digest-bound in the second run's record, GitLab CE 17.11.7 commits 8 transitions over 432 cells, with entitlement and effect agreeing on 432 of them: 16 stale false proofs, all 16 removed by a recrawl with the same credentials, 0 live false proofs and 0 residual misses. Pooled with Gitea this is 712 cells, 712 agreements, 29 stale false proofs and 29 fixes. The replication is of a protocol, not of a topology: GitLab's project-membership surface answers 200 or 404, so it errs only by omission—in 53 cells it disagrees with the ledger, every one of them a false deny, concentrated in the public (44) and site-administration (9) channels—and both properties are read from the same run rather than one being derived from the other. The entitlement/effect gate is in turn checked against a real defect: on a publicly fixed object-versus-path ownership flaw, Gitea 1.20.5 yields 1 true positive with 0 false proofs, and the fixed 1.20.6 yields 0 alarms over 10 control probes. That is a check of the gate on one defect we located and supplied; it is not a detection rate.

How far the claims reach

The native run establishes a qualitative outcome, for one gate family, on one real system that on the tested endpoints and transitions enforces the expected decisions; it does not measure the accuracy of MST-wi as a detector. Three ceilings stack and bound everything above. First, our two relations are two of a catalogue of 76, and both belong to one reachability-gated family. Second, even the execution of the published relations on the native engine reaches only that family: 2 of the 11 authorization relations fire (10 executed; OTG_AUTHZ_002, OTG_AUTHZ_002c), and the content predicate `userCanRetrieveContent` is never invoked (0 calls), so that evidence is bounded the same way. Third, the real-system study exercises the *false-proof* face on a single constructed fault pattern, not the detection face against a population of real flaws, and the Gitea channel proportions are topology design parameters, not field prevalence. Together these fix the ceiling on what a practitioner may transfer.

What a practitioner should do differently

Three operational consequences follow, each bounded by what was measured. Do not read a reachability-gated relation's alarm as proof: because the reading time is part of the specification, an alarm raised against a crawl-time snapshot—rather than against live state—cannot be promoted to `violation-proven` on the relation's word alone, and

the policy that decides promotion should be reported with the count it produces (Table 21). If an oracle is readable, sound *and* passes the independence admission test of Section VI-F, consult it directly rather than through the relation: on the domain where it agrees with the ledger the relation adds no confirmation. A readable, sound oracle that *fails* that test does not license the substitution, and none of the three products studied passes it—substituting there raises false proofs from 8 to 17 on Gogs and from 5 to 18 on GitLab, so on the systems we could test the advice to drop the relation is unlicensed. And treat a reported input-level specificity as a corpus statistic rather than a guarantee of scenario-level resolution: it can sit near one while a whole class of legitimate access is misreported (Finding 3). They locate metamorphic testing's contribution where no oracle exists, and bound it by what the reachability reading can see.

What a stronger claim would require

Moving from characterization to something stronger needs, first, a population of real systems that *contain* the flaw, so detection can be measured—neither the four seeded classes of Section V-D nor the single publicly fixed defect of Section V-E supplies that population: the first are planted in an emulator, the second is a real defect replayed under a controlled topology, and neither is a sample of defects found in the field; and second, relations whose gate is not the absence of evidence—checkable against a subject's entitlements without pretending that entitlements are reachable facts. The transferable condition—we call it the *same-observable condition*—is the one that killed our own mechanism: a component's necessity must be tested against a substitute that consumes the *same* observable quantity as the ground truth; a substitute that consumes less does not establish incremental value. The MST-wi case study is an instance of that condition, not its scope: a component's necessity is not established unless a fair control that lacks the component but consumes the same decision-relevant observables is empirically or formally shown to be weaker.

DECLARATIONS

DATA AVAILABILITY

The raw records of the studies reported here are publicly available in the companion repository https://github.com/ubfbuz3/JISA, version-pinned by the tag `access-resubmission-v18`, which archives every record behind every number in this paper, the frozen admission protocol, its errata and the application register (`real_system/prereg/`), and the generation scripts that recompute the tables from those records, including the scoring-rule, ledger-omission and promotion-policy sensitivities (Tables 1, 2 and 21). A previous submission is pinned separately by `access-resubmission-v12`; it carries the records behind the earlier analyses, not the analyses added here and not the paired recrawl experiment of Section V-A, whose record (`results/r15_pair.json`) was produced for this version. Earlier tags (`artifact-v1` and `access-resubmission-v2` through `v17`) are retained as snapshots of prior states and should not be conflated with either. The records are organised under `experiments/`: (1) the scenario/configuration study (`results/raw_matrix.jsonl`, `ablation_direct*.json`, `alt_matrix.jsonl`); (2) the real-system cell study across Gitea 1.22.6, Gogs 0.14.3 and GitLab 17.11.7 (`real_system/results/*.json`), including the three-arm paired recrawl (`r15_pair.json`); (3) the temporal loop and its archived pilot, with the protocol and both amendments by digest (`real_system/results/temporal/`, `c_protocol*.json`); (4) the real vulnerable/fixed pair (`real_system/results/cve_pair.json`); (5) the execution of the published relations on the native engine (`native_engine/results/*.json`); (6) the seeded positive control (`seeded.json`); (7) the equal-budget sweep with its per-cell records (`results/baseline_budget.json`); and (8) the four-class defect study with per-instance and per-channel negative records (`real_system/results/seeded_multiclass.json`). The analysis pipeline (`analysis/*.py`) recomputes every number, table and figure from those records, including the design audit (`ingest_design.py`), the decision-scored coverage table (`ingest_coverage.py`), the truth table (`ingest_truth.py`) and the caliber, ledger and policy sensitivities of Tables 1, 2 and 21 (`ingest_revision.py`); `check_macro_use.py` asserts that every macro cited in the manuscript has exactly one definition. `bash reproduce.sh` regenerates all outputs and the PDF, and `bash verify_artifact.sh` checks the SHA-256 of every raw record and of every byte-reproducible output. The manuscript LaTeX source ships in the same repository. The DOI https://doi.org/10.5281/zenodo.22859942 archives an earlier release of this artifact; a Zenodo version covering the present one will be deposited at acceptance, and the git tag above is the version-pinned copy in the meantime.

COMPETING INTERESTS

The authors declare no competing interests, financial or otherwise.

FUNDING

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

AUTHORS' CONTRIBUTIONS

Qiulong Lin, Xiaoyu Ke, and Nanlei Chen contributed to the work. Qiulong Lin and Nanlei Chen jointly developed the conceptualisation and methodology, designed and implemented the software and analysis pipeline, carried out the investigation and formal analysis, and wrote the manuscript. Xiaoyu Ke guided the experimental work and reviewed the experiments and their analysis.

ETHICS

Not applicable. This study involved no human participants, no personal data, and no animals. All experiments were performed against software systems that we deployed and controlled locally.

DECLARATION OF GENERATIVE AI AND AI-ASSISTED TECHNOLOGIES IN THE WRITING PROCESS

During the preparation of this work the authors used Claude (Anthropic) to assist with literature verification, the development of the analysis scripts, and language editing. Where it contributed to analysis code, every generated or modified script was reviewed, executed, and its output checked by the authors against the raw records. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. All experimental computations and final reported values were independently checked by the authors against the released scripts and raw records.

REFERENCES

[1] OWASP Foundation, "API1:2023 Broken Object Level Authorization," OWASP API Security Top 10, 2023, accessed 18 September 2026. [Online]. Available: https://api-security.owasp.org/editions/2023/en/0xa1-broken-object-level-authorization/

[2] N. B. Chaleshtari, F. Pastore, A. Goknil, and L. Briand, "Metamorphic testing for web system security," *IEEE Transactions on Software Engineering*, pp. 3430–3471, 2023. [Online]. Available: https://doi.org/10.1109/TSE.2023.3256322

[3] P. X. Mai, F. Pastore, A. Goknil, and L. Briand, "Metamorphic security testing for web systems," in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, 2020, pp. 186–197. [Online]. Available: https://doi.org/10.1109/icst46399.2020.00028

[4] R. Croft, M. A. Babar, and M. M. Kholoosi, "Data quality for software vulnerability datasets," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 121–133. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00022

[5] F. Liu, Y. Zhang, E. Li, W. Meng, Y.-Q. Shi, Q. Wang, and M. Yang, "Bacscan: Automatic black-box detection of broken-access-control vulnerabilities in web applications," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, 2025, pp. 1320–1333. [Online]. Available: https://doi.org/10.1145/3719027.3744825

[6] I. P. A. Dharmaadi, M. Alhanahnah, V.-T. Pham, F. Mohsen, and F. Turkmen, "Bacfuzz: Exposing the silence on broken access control vulnerabilities in web applications," *ArXiv*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2507.15984

[7] C. Zuo, Q. Zhao, and Z. Lin, "Authscope: Towards automatic discovery of vulnerable authorizations in online services," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 799–813. [Online]. Available: https://doi.org/10.1145/3133956.3134089

[8] S. E. H. Chehade, F. Hantke, and B. Stock, "403 forbidden? ethically evaluating broken access control in the wild," in *2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 3218–3235. [Online]. Available: https://doi.org/10.1109/SP61157.2025.00252

[9] A. S. Filho, R. J. Rodríguez, and E. Feitosa, "Automated broken object-level authorization attack detection in rest apis through openapi to colored petri nets transformation," *International Journal of Information Security*, vol. 24, no. 2, 2025. [Online]. Available: https://doi.org/10.1007/s10207-024-00970-5

[10] H. Le, L. K. Shar, D. Bianculli, L. Briand, and D. C. Nguyen, "Automated reverse engineering of role-based access control policies of web applications," *Journal of Systems and Software*, vol. 184, p. 111109, 2022. [Online]. Available: https://doi.org/10.1016/j.jss.2021.111109

[11] B. Berger, R. W. Nguempnang, K. Sohr, and R. Koschke, "Static extraction of enforced authorization policies seeauthz," in *2020 IEEE 20th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2020, pp. 187–197. [Online]. Available: https://doi.org/10.1109/SCAM51674.2020.00026

[12] W. Zhang, D. Bali, J. Kerney, A. Panda, and S. Shenker, "Extracting database access-control policies from web applications," in *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI '26)*, 2026, pp. 249–270. [Online]. Available: https://www.usenix.org/conference/osdi26/presentation/zhang-wen

[13] Y.-Q. Shi, F. Liu, G. Yang, Y. Zhang, Y. Cao, E. Li, and S. Chen, "Facilitating access control vulnerability detection in modern java web applications with accurate permission check identification," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 10 683–10 698, 2025. [Online]. Available: https://doi.org/10.1109/TIFS.2025.3614424

[14] Y. Huang, C. Shi, J. Lu, H. Li, H. Meng, and L. Li, "Detecting broken object-level authorization vulnerabilities in database-backed applications," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 2934–2948. [Online]. Available: https://doi.org/10.1145/3658644.3690227

[15] F. Liu, Y.-Q. Shi, Y. Zhang, G. Yang, E. Li, and M. Yang, "Mocguard: Automatically detecting missing-owner-check vulnerabilities in java web applications," in *2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 903–919. [Online]. Available: https://doi.org/10.1109/SP61157.2025.00010

[16] R. Feng, M. Zhang, A. Xia, J. Cheng, Y. Li, Y. Zhang, and Y. Liu, "Jauthguard: Automatic detection for broken access control in java web apis," in *2025 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 2025, pp. 3102–3107. [Online]. Available: https://doi.org/10.1109/SMC58881.2025.11343625

[17] M. Schlaubitz, O. Veyisoglu, and M. Rennhard, "A2ct: Automated detection of function and object-level access control vulnerabilities in web applications," in *International Conference on Information Systems Security and Privacy*, 2025, pp. 425–436. [Online]. Available: https://doi.org/10.5220/0013092700003899

[18] P. Li, H. Chong, Y. Cao, and J. Yang, "Detecting privilege escalation in polyglot microservices via agentic program analysis," in *2026 IEEE Symposium on Security and Privacy (SP)*, 2026, pp. 1747–1765. [Online]. Available: https://doi.org/10.1109/SP63933.2026.00121

[19] V. Atlidakis, P. Godefroid, and M. Polishchuk, "Restler: Stateful rest api fuzzing," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, 2019, pp. 748–758. [Online]. Available: https://doi.org/10.1109/ICSE.2019.00083

[20] A. Arcuri, M. Zhang, and J. P. Galeotti, "Advanced white-box heuristics for search-based fuzzing of rest apis," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 6, pp. 1–36, 2024. [Online]. Available: https://doi.org/10.1145/3652157

[21] S. Seran, M. Zhang, O. Duman, and A. Arcuri, "Handling web service interactions in fuzzing with search-based mock-generation," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 2, pp. 1–45, 2026. [Online]. Available: https://doi.org/10.1145/3731558

[22] Ömür Şahin, M. Zhang, and A. Arcuri, "Enhancing rest api fuzzing with access policy violation checks and injection attacks," *Journal of Systems and Software*, vol. 243, p. 113060, 2027. [Online]. Available: https://doi.org/10.1016/j.jss.2026.113060

[23] M. A. Uddin, S. Weerasinghe, C. Wojtak, T. Cerný, D. Silva-Junior, M. E. S. Ribeiro, and A. S. Abdelfattah, "Automated generation of microservice authorization tests using large language models," in *2026 International Conference on Service-Oriented System Engineering (SOSE)*, 2026, pp. 41–50. [Online]. Available: https://doi.org/10.1109/SOSE71128.2026.00014

[24] J. Sun, R. Tian, S. Zhao, X. Zhou, H. Wang, and Y. Zhang, "Llm-assisted idor detection in hospital mini-programs: Risks to pii and phi," in *2025 IEEE 24th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, 2025, pp. 2078–2086. [Online]. Available: https://doi.org/10.1109/Trustcom66490.2025.00242

[25] E. Wang, W. Xie, S. Li, R. Liu, Y. Zhou, Z. Wang, and B. Wang, "Large language model-powered protected interface evasion: Automated discovery of broken access control vulnerabilities in internet of things devices," *Sensors (Basel, Switzerland)*, vol. 25, no. 9, p. 2913, 2025. [Online]. Available: https://doi.org/10.3390/s25092913

[26] A. Wu, Z. Feng, R. Feng, Z. Xing, and Y. Liu, "Rethinking broken object level authorization attacks under zero trust principle," *ACM Transactions on Software Engineering and Methodology*, 2026. [Online]. Available: https://doi.org/10.1145/3833417

[27] H. Wang, Z. Wang, and P. Liu, "A hybrid llm workflow can help identify user privilege related variables in programs of any size," *ArXiv*, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2403.15723
[28] C. Wen, J. Cao, J. Su, Z. Xu, S. Qin, M. He, and C. Tian, "Enchanting program specification synthesis by large language models using static analysis and program verification," in *International Conference on Computer Aided Verification*, 2024, pp. 302–328. [Online]. Available: https://doi.org/10.1007/978-3-031-65630-9_16
[29] R. Wang, H. Zhuang, H. Cao, and S. Chen, "Measuring the reasoning boundaries of large language models for implicit security invariants in code: A controlled empirical study," in *2026 IEEE 50th Annual Computers, Software, and Applications Conference (COMPSAC)*, 2026, pp. 3146–3157. [Online]. Available: https://doi.org/10.1109/COMPSAC69091.2026.00467
[30] T. Le-Cong, B. Le, and T. Murray, "Can llms reason about program semantics? a comprehensive evaluation of llms on formal specification inference," in *Annual Meeting of the Association for Computational Linguistics*, 2025, pp. 21 991–22 014. [Online]. Available: https://aclanthology.org/2025.acl-long.1068/
[31] J. Liu, G. Lin, H. Mei, F. Yang, and Y. Tai, "Enhancing vulnerability detection efficiency: An exploration of light-weight llms with hybrid code features," *Journal of Information Security and Applications*, vol. 88, p. 103925, 2024. [Online]. Available: https://doi.org/10.1016/j.jisa.2024.103925
[32] J. Senanayake, H. Kalutarage, A. Petrovski, L. Piras, and M. O. Al-Kadri, "Defendroid: Real-time android code vulnerability detection via blockchain federated neural network with xai," *Journal of Information Security and Applications*, vol. 82, p. 103741, 2024. [Online]. Available: https://doi.org/10.1016/j.jisa.2024.103741

QIULONG LIN Qiulong Lin received the M.S. degree from Sogang University, Seoul, Republic of Korea. He is with Yango University, Fuzhou, China, and is also affiliated with Sogang University, Seoul, Republic of Korea. His research interests include Web/API security, vulnerability discovery, and metamorphic testing.

XIAOYU KE Xiaoyu Ke is with Yango University, Fuzhou, China.

NANLEI CHEN Nanlei Chen is currently pursuing the M.S. degree at Sogang University, Seoul, Republic of Korea. He is with Yango University, Fuzhou, China. His research interests include Web/API security and authorization testing. He is the corresponding author of this work.